TeaRAG: A Token-Efficient Agentic Retrieval-Augmented Generation Framework

CHAO ZHANG, University of Science and Technology of China and City University of Hong Kong, China
YUHAO WANG, City University of Hong Kong, China
DERONG XU, University of Science and Technology of China and City University of Hong Kong, China
HAOXIN ZHANG, Xiaohongshu Inc., China
YUANJIE LYU, YUHAO CHEN, SHUOCHEN LIU, and TONG XU*, University of Science and Technology of China, China
XIANGYU ZHAO*, City University of Hong Kong, China
YAN GAO and YAO HU, Xiaohongshu Inc., China
ENHONG CHEN, University of Science and Technology of China, China

Retrieval-Augmented Generation (RAG) utilizes external knowledge to augment Large Language Models' (LLMs) reliability. For flexibility, agentic RAG employs autonomous, multi-round retrieval and reasoning to resolve queries. Although recent agentic RAG has improved via reinforcement learning, they often incur substantial token overhead from search and reasoning. This trade-off prioritizes accuracy over efficiency. To address this issue, this work proposes TeaRAG, a **T**oken-**e**fficient **a**gentic **RAG** framework capable of compressing both retrieval content and reasoning steps. 1) First, the retrieved content is compressed by augmenting chunk-based semantic retrieval with a graph retrieval using concise triplets. A knowledge association graph is then built from semantic similarity and co-occurrence. Finally, Personalized PageRank is leveraged to highlight key knowledge within this graph, reducing the number of tokens per retrieval. 2) Besides, to reduce reasoning steps, Iterative Process-aware Direct Preference Optimization (IP-DPO) is proposed. Specifically, our reward function evaluates the knowledge sufficiency by a knowledge matching mechanism, while penalizing excessive reasoning steps. This design can produce high-quality preference-pair datasets, supporting iterative DPO to improve reasoning conciseness. Across six datasets, TeaRAG improves the average Exact Match by 4% and 2% while reducing output tokens by 61% and 59% on Llama3-8B-Instruct and Qwen2.5-14B-Instruct, respectively. Code is available at https://github.com/Applied-Machine-Learning-Lab/TeaRAG.

CCS Concepts: • **Information systems → Information retrieval**; • **Computing methodologies → Natural language generation**.

Additional Key Words and Phrases: Agentic Retrieval-Augmented Generation, Token-Efficient, Knowledge Graph, Process Supervision

*Corresponding authors.

Authors' Contact Information: Chao Zhang, zclfe00@mail.ustc.edu.cn, University of Science and Technology of China and City University of Hong Kong, China; Yuhao Wang, City University of Hong Kong, China, yhwang25-c@my.cityu.edu.hk; Derong Xu, derongxu@mail.ustc.edu.cn, University of Science and Technology of China and City University of Hong Kong, China; Haoxin Zhang, Xiaohongshu Inc., China, zhanghaoxin1994@gmail.com; Yuanjie Lyu, s1583050085@gmail.com; Yuhao Chen, isyuhaochen@mail.ustc.edu.cn; Shuochen Liu, shuochenliu@mail.ustc.edu.cn; Tong Xu, tongxu@ustc.edu.cn, University of Science and Technology of China, China; Xiangyu Zhao, xianzhao@cityu.edu.hk, City University of Hong Kong, China; Yan Gao, yadun@xiaohongshu.com; Yao Hu, yaoohu@gmail.com, Xiaohongshu Inc., China; Enhong Chen, cheneh@ustc.edu.cn, University of Science and Technology of China, China.

ACM 1558-2868/2026/6-ART
https://doi.org/10.1145/3818621

1 Introduction

Large Language Models (LLMs) have made substantial progress through learning from massive pre-training corpora. Nevertheless, statistical inaccuracies in linguistic distribution modeling during pre-training can lead to hallucination generation [30]. Retrieval-Augmented Generation (RAG) is an effective technique that mitigates hallucinations in LLMs by incorporating external retrieval information [13, 43]. To improve the accuracy of RAG, conventional RAG systems typically adopt a predefined multi-step workflow consisting of planning [66], query rewriting [47, 89], retrieval [44], reranking [70], refinement [24], and generation. However, the fixed workflows constrain their capacity to address tasks that demand multi-step reasoning, adaptive decision-making, and dynamic integration of retrieved information [37, 44]. To further enhance the flexibility and adaptiveness of the RAG framework, agentic RAG has recently been proposed. Agentic RAG adopts agentic workflows that allow LLMs to autonomously control the workflow process to solve complex problems [44, 59]. By leveraging the capabilities of LLMs in task decomposition and dynamic planning, agentic RAG can break down problems into a sequence of tractable steps. At each step, the framework proactively invokes retrieval modules to obtain relevant contextual information, thereby establishing an integration between reasoning and retrieval [28, 64].

Recently, the performance of agentic RAG has been significantly enhanced through reinforcement learning (RL) optimization of agentic workflows for LLMs [58, 59, 61]. For example, Search-R1 [28] employs PPO [56] and GRPO [57] to optimize agentic RAG based on an output-based reward. However, the current paradigm still faces several challenges. First, existing agentic RAG methods are token-inefficient. This is because current systems predominantly prioritize maximizing the accuracy of the final outputs, while overlooking the substantial token overhead during the reasoning and retrieval processes [28, 58]. This optimization bias often leads models to overthink [7, 9] and perform redundant retrieval [71], resulting in wasted computational resources and reduced system efficiency. Second, current agentic RAG systems largely rely on chunk-based semantic retrieval, which yields low information density and tends to introduce irrelevant noise [44, 84]. While recent studies [42, 82] have investigated the use of higher-density knowledge graphs for retrieval, it remains unvalidated on large-scale graphs and fails to leverage co-occurrence relationships among chunks and knowledge triplets. This prevents the system from leveraging both retrieval methods' strengths [17] and hinders retrieval systems in preserving critical information and removing irrelevant noise. Finally, most agentic RAG systems train LLMs with outcome-based rewards via RL [28, 59]. This reward design is typically sparse and noisy, which can hinder stable and efficient training [71]. Besides, agentic RAG methods [58, 71] are usually trained using RL algorithms such as PPO [56] and GRPO [57]. These approaches require multi-machine coordination and continuous external retrieval access during training [61], which makes the training process prohibitively slow. To address this limitation, recent approaches employ GPT-4o for process-based reward annotation [88], and then optimize using DPO [54]. However, the reward annotation process is costly and depends on proprietary models.

To gain deeper insights into the token usage of existing agentic RAG systems, we analyze two representative methods: Search-R1 [28] and R1-Searcher [59]. As illustrated in Fig. 1 (a), the output of an agentic RAG system primarily consists of two types of tokens generated through multiple iterations. The first is the LLM's thinking process, where these tokens are used for planning, problem decomposition, and reasoning over retrieved content. The second is the retrieved content that the LLM obtains by invoking a retriever to access external sources. Through statistical analysis of different token types and iteration rounds, as observed in Fig. 2 (a) and Fig. 2 (b), we identify two critical inefficiencies that impact token utilization. First, the retrieved content constitutes the majority of the overall output. This is because chunk-based retrieval methods typically return entire document segments as input to LLMs. However, a substantial portion of these contents may consist of irrelevant or redundant background information that does not enhance the quality of the final answer [44]. Therefore, increasing the information density of the retrieved content is essential for improving the token efficiency in agentic RAG. Second, agentic RAG methods generally adopt multi-step reasoning, even when addressing single-hop questions.

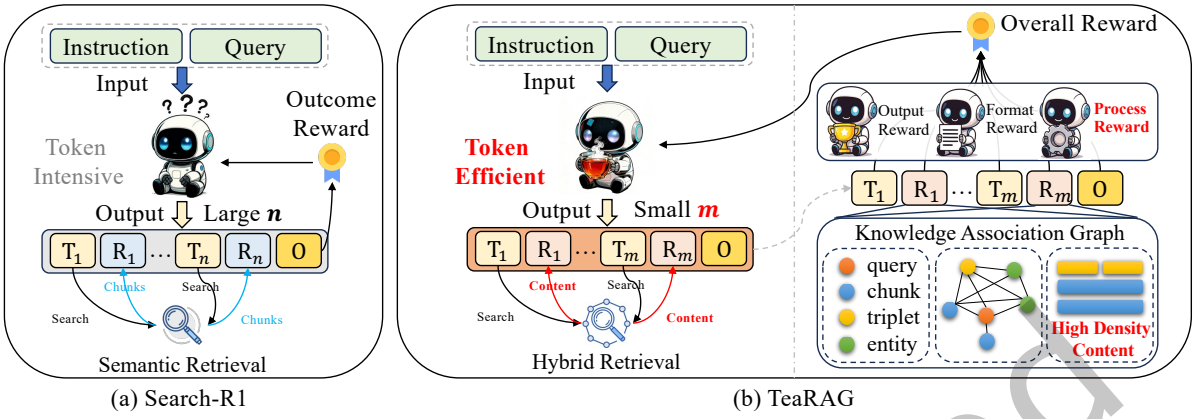


Fig. 1. T_i denotes the thinking tokens at the i-th step, R_i denotes the retrieved context at the i-th step, and O represents the final output. (a) illustrates Search-R1, a representative agentic RAG method optimized based on the final outcome. (b) shows our proposed method TeaRAG, which achieves a token-efficient agentic RAG by optimizing the retrieved content length with high-density triplets and controlling the number of LLM reasoning steps via a process-aware reward.

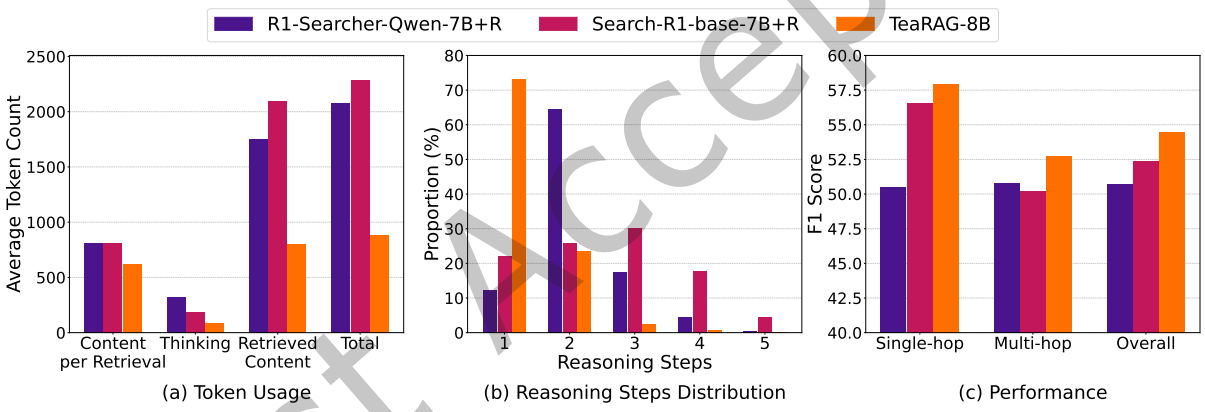


Fig. 2. (a) shows the token usage. (b) shows the distribution of reasoning steps. (c) shows the F1 performance on single-hop, multi-hop, and overall QA benchmarks.

As shown in Fig. 2 (b), single-hop questions account for 44% of the test set, yet in most cases the number of reasoning steps exceeds one. This is due to the lack of process supervision in outcome-based rewards, which leads to overthinking [7] and redundant retrieval [67]. Therefore, reducing unnecessary and repetitive steps in intermediate processes is key to improving the efficiency of token utilization.

To address these challenges, we introduce TeaRAG, a token-efficient agentic RAG framework that enhances token efficiency by simultaneously optimizing the conciseness of reasoning steps and the density of retrieved content, as shown in Fig. 1 (b). Specifically, the workflow of this agentic RAG system is autonomously controlled by an LLM. First, the LLM identifies key entities in the question and breaks it down into sub-questions around these key entities. Subsequently, based on the sub-questions, the LLM calls a retriever to perform semantic retrieval at the chunk level. To improve the density of the retrieved content, the LLM also uses graph retrieval to retrieve

relevant and concise knowledge triplets. Afterwards, based on the recalled chunks and triplet information, we construct a Knowledge Association Graph (KAG) using semantic similarity and co-occurrence. Using Personalized PageRank (PPR) on this graph, we filter out redundant and irrelevant chunk information while supplementing it with the higher-density triplet information. Finally, the LLM summarizes the retrieved content. The LLM repeats this process until the final answer is obtained.

To effectively implement this pipeline and improve the conciseness of LLM reasoning, we propose a two-stage training paradigm. In the first stage, we construct supervised fine-tuning (SFT) data to train models to master the reasoning format and thinking process. Specifically, we leverage the query decomposition processes from the MuSiQue [63] dataset and employ Qwen2.5-72B-Instruct [79] to transform structured question-answer pairs into natural language equivalents. We then assemble these into complete reasoning processes following chain combinations. Using this SFT dataset, we perform SFT on the models. In the second stage, we propose Iterative Process-aware Direct Preference Optimization (IP-DPO), which incorporates a novel process reward and iteratively optimizes LLMs. Specifically, we sample multiple reasoning paths for each query, assign rewards to them, and construct preference-pair datasets based on these rewards. For a given sampled reasoning path, the reward system integrates conventional output-based and format-based rewards with the proposed process reward to produce a comprehensive evaluation score. The process reward employs a knowledge matching mechanism to assess evidence acquisition across three dimensions: subquery generation, context retrieval, and summarization. This mechanism measures the alignment of intermediate reasoning outputs in each dimension with ground truth knowledge. Then, the process reward is computed by aggregating evidence acquisition scores from each dimension and normalizing by the total reasoning steps, resulting in the information gain per step. Besides, the process reward also ensures consistency between extracted entities and subqueries to facilitate the PPR filtering. Based on this reward framework, we can create high-quality preference-pair datasets, enabling iterative DPO to improve model generalization and yield more compact reasoning paths. Comprehensive evaluations on six benchmark datasets demonstrate that TeaRAG attains superior accuracy while reducing both reasoning steps and output length as shown in Fig. 2. On Llama3-8B-Instruct, it improves the average Exact Match (EM) score by 4% alongside a 61% reduction in output tokens, whereas on Qwen2.5-14B-Instruct it yields a 2% EM increase with a 59% token reduction.

Our contributions can be summarized as follows:

- We conduct an in-depth analysis of token inefficiencies in agentic RAG and propose a token-efficient pipeline, TeaRAG, that simultaneously improves the information density of retrieved contents and reduces reasoning steps.
- We propose a retrieval method that constructs KAGs via semantic similarity and co-occurrence, combining semantic and graph retrieval strengths. By employing PPR filtering to remove irrelevant contents, our approach retrieves more concise contents without sacrificing performance.
- We propose a two-stage training paradigm that effectively activates agentic RAG reasoning capabilities while maintaining reasoning conciseness through process-aware training.
- We conduct comprehensive experiments and detailed analyses across six benchmark datasets, validating the effectiveness and token efficiency of our pipeline and training methods.

2 Related Work

2.1 Retrieval-Augmented Generation

LLMs are trained on massive datasets, possessing powerful capabilities in understanding [13], generation [79], and reasoning [37, 57]. However, when faced with knowledge-intensive tasks [36], LLMs that rely solely on their parametric memory are prone to hallucination [13, 28, 35]. RAG addresses this by using a retriever to search a corpus for information snippets relevant to a query. These information are then provided to the LLM as

evidence to generate more reliable answers. The field of RAG has attracted significant attention in recent years, spurring substantial research. We will focus on introducing agentic RAG and graph-enhanced RAG, which are most relevant to our work.

2.1.1 Agentic RAG. A comprehensive RAG system typically involves multiple sub-steps, including query planning [66], query rewriting [38, 47], retrieval [44], reranking [70], refinement [24], and generation. Traditional RAG systems usually follow pre-defined sub-steps that are manually designed by humans. However, this approach fails to dynamically adapt to task requirements and performs poorly when facing complex multi-hop tasks [16, 40, 90]. To autonomously control the RAG process, agentic RAG applies the agent approach [41, 68, 81]. Through the planning and comprehension capabilities of LLMs, it iteratively breaks down complex problems into executable operations and queries until an answer is found.

Initial works use prompt-based approaches for preset process control [37, 64, 86], determining whether RAG should continue querying or provide an answer to the question. However, due to the lack of training specifically for RAG scenarios, these methods rely on LLMs' instruction-following and reasoning capabilities. To enhance the adaptability of LLMs to RAG tasks, some studies [2, 44] employ SFT to train agentic RAG systems. However, such approaches often fall short in further stimulating the generalization capability of the models. Recently, as RL methods have greatly enhanced the reasoning and generalization capabilities of LLMs [57], numerous studies have employed outcome-based RL to stimulate LLMs' thinking and tool-use abilities [28, 59]. These works typically use string matching methods to simply determine whether the LLM's output is consistent with the ground truth [26, 28, 59] or whether the LLM's reasoning process contains the ground truth [58]. Nevertheless, such approaches often suffer from suboptimal reasoning processes due to the lack of supervision over intermediate steps [71]. Some recent work has explored process-reward methods to more precisely optimize agentic RAG systems [71, 88]. These approaches include employing predefined reasoning format rewards to supervise the model in learning correct reasoning formats [27, 60] and designing reward mechanisms based on the number of retrieval calls [60, 82]. Further improvements involve leveraging more comprehensive evidence paragraphs to supervise the reasoning process [71], or utilizing GPT-4o to annotate intermediate reasoning steps [88].

Although current agentic RAG has made significant progress, existing methods focus on training LLMs on how to invoke retrievers for better performance, often neglecting token efficiency. Besides, these approaches either rely on high-resource RL methods such as PPO [56] and GRPO [57], which require multi-machine collaboration and continuous retrieval resource requests during training [71], or utilize GPT-4o for process scoring [88]. Both approaches make it difficult to achieve efficient, low-resource implementation of agentic RAG. Therefore, our work aims to improve the token efficiency of agentic RAG by reducing token usage while maintaining performance. Additionally, we propose an iterative process-aware DPO to implement agentic RAG.

2.1.2 Graph-Enhanced RAG. Most traditional RAG relies on chunk-based retrieval approaches [10], which split complete document information into multiple chunks and use these chunks as the units for retrieval and input to LLMs. However, since question-related clues are embedded in complex chunk contexts, existing chunk-based approaches struggle to accurately capture subtle clues [15, 44] and multi-hop knowledge associations [16, 17]. Previous work organizes corpus knowledge into knowledge graph structures to capture fine-grained clues and knowledge associations more systematically [10, 15, 76, 77]. Leveraging the topological structure of knowledge graphs, many works replace chunk-based retrieval methods with graph algorithms to achieve more accurate results [6, 11, 16, 17, 75]. These graph-enhanced RAG approaches can be categorized into three strategies. First, some methods employ classical graph algorithms such as Prize-Collecting Steiner Tree [18] and PPR [1, 16, 17] to retrieve all query-related information in one step, then feed the entire retrieved context to the LLM for direct answer generation [75, 91]. Second, some approaches employ LLM prompts to select the optimal triplet path from all candidate reasoning paths based on topological relationships in the graph structure [6, 11, 62]. Last, recent concurrent work has explored extending RL-based training approaches from agentic RAG to graph-enhanced

RAG, aiming to fully leverage the reasoning capabilities of LLMs [42, 82]. Nevertheless, a key challenge in knowledge graph construction is the potential for information loss [74], as the process simplifies complete text into structured triplets. To address this issue, some works dynamically construct knowledge graphs after receiving specific queries [25, 78], which enables the targeted capture of core information relevant to the query from the text. Nevertheless, these approaches incur additional online inference time and high computational costs. Alternatively, some methods adopt a hybrid approach that combines chunk-based semantic retrieval with graph-based retrieval mechanisms to leverage the advantages of both strategies [17, 39, 46, 55, 82].

Current graph-enhanced RAG still has its limitations. First, these methods typically utilize existing small-scale knowledge graphs [18, 75] or construct them from relatively small (sub-million scale) corpora [16, 17, 42], lacking validation on sufficiently large-scale graph data. Furthermore, graph-enhanced RAG systems typically select a fixed number of top-ranked triplets [6, 11, 75] or their associated text chunks [16, 17], or directly concatenate the retrieved triplets and chunks as input [55, 82]. However, these approaches fail to fully exploit the co-occurrence of different representations of the same knowledge element. In our work, we conduct experimental validation by constructing a large-scale knowledge graph based on a common wiki corpus [31]. We use PPR on a KAG of retrieved chunks and knowledge triplets to select the most relevant information. This process boosts information density while preserving key context, achieving a token-efficient agentic RAG.

2.2 Reinforcement Learning for LLMs

LLMs gain knowledge through pre-training on vast data [3, 79] and are then instruction-tuned via SFT to follow instructions [8, 14, 50]. However, this imitation-based learning limits their ability to generalize. To address these limitations, Reinforcement Learning from Human Feedback [50] (RLHF) first trains a reward model to capture human preferences over model outputs. This reward model then guides the LLM optimization using PPO [56], enabling the generation of higher-quality, more aligned responses. Beyond alignment with human values, RL techniques have proven effective in enhancing LLM reasoning capabilities [23, 45, 57]. For example, methods like GRPO [57] improve performance on mathematical reasoning tasks by optimizing the model's generation process using either outcome-based rewards [57, 83] or fine-grained process reward models that provide step-by-step supervision [87]. However, the above methods are typically online RL approaches, requiring LLMs to perform sampling simultaneously during training [28, 57], which often leads to low training efficiency when LLM inference is lengthy or requires calling complex tools [12, 54]. By separating the sampling and training phases, easier and more efficient RL methods achieve higher training efficiency, sometimes at the expense of performance [22]. These include rejection sampling [32, 73, 85], which enhance model performance by filtering high-quality samples from generated data for subsequent SFT. DPO [32, 54] circumvents explicit reward model training by treating the LLM itself as an implicit reward model, enabling RL through preference pair construction. This approach has been successfully applied to reasoning tasks through iterative sampling and training cycles [52, 65]. Building on this, our work utilizes RL algorithms to optimize the multi-turn retrieval and reasoning capabilities of agentic RAG systems. We design novel process-based rewards that employ knowledge matching to evaluate key components throughout the LLM's reasoning paths. These rewards effectively distinguish between positive and negative samples, and we employ multi-round iterative DPO to train LLMs for adaptation to agentic RAG tasks efficiently.

3 Preliminaries

Given a task instruction I, a question q, a retriever $\mathcal{R}$, a collection of document chunks $\mathcal{D} = \{d_1, \ldots, d_n\}$, and a knowledge graph $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$ where $\mathcal{V} = \{v_1, \ldots, v_m\}$ is the entity set and $\mathcal{E} = \left\{ e_i \,|\, e_i = (v_i^h, rel, v_i^t),\ v_i^h, v_i^t \in \mathcal{V} \right\}$ is the triplet set, RAG utilizes the retriever $\mathcal{R}$ to retrieve relevant information based on question q and inputs it into an LLM for reasoning and answering.

Definition 3.1. **Single-Round RAG.** This paradigm involves a single retrieval step for a given query q. A retrieval algorithm $\mathcal{A}_{\mathcal{R}}$, using a retriever $\mathcal{R}$, gathers information from a document corpus $\mathcal{D}$ and a knowledge graph $\mathcal{G}$. The resulting context, $\mathcal{C}_q = \mathcal{A}_{\mathcal{R}}(q, \mathcal{D}, \mathcal{G})$, is the union of retrieved document chunks ($\mathcal{D}_q \subseteq \mathcal{D}$) and knowledge graph triplets ($\mathcal{E}_q \subseteq \mathcal{G}$). This aggregated context is then combined with the instruction I and query q to form the final prompt for the LLM, which generates the answer a: $a = \text{LLM}(I, q, \mathcal{C}_q)$.

Definition 3.2. **Agentic RAG.** This paradigm empowers the LLM to act as an autonomous agent that constructs a dynamic workflow to answer complex questions. The agent iteratively builds a reasoning path, $\mathcal{P}_k$, by interleaving thought and retrieved context. Each step i of the process unfolds in two parts. First, in the thinking phase, the LLM generates an internal reasoning step $T_i = \text{LLM}(I, q, \mathcal{P}_{i-1})$ based on the instruction I, question q, and its previous reasoning steps. Second, in the retrieval phase, a subquery q_i is extracted from the thought T_i and used to retrieve relevant context $\mathcal{C}_{q_i} = \mathcal{A}_{\mathcal{R}}(q_i, \mathcal{D}, \mathcal{G})$. This cycle continues until a complete reasoning path, $\mathcal{P}_k = [T_1, \mathcal{C}_{q_1}, \ldots, T_k, \mathcal{C}_{q_k}]$, is constructed. Finally, the LLM generates the final answer based on this complete path: $a = \text{LLM}(I, q, \mathcal{P}_k)$.

This work aims to develop a token-efficient agentic RAG system that preserves correctness while compressing $\mathcal{P}_k$, thereby enhancing token efficiency. As analyzed in Section 1, TeaRAG achieves this goal via two key perspectives. (1) constructing higher-density contexts $\mathcal{C}_{q_i}$ by shortening the context per retrieval while retaining essential content. (2) reducing the number of reasoning steps k thereby enhances the efficiency of reasoning.

4 Methodology

In this section, we present the overall implementation of TeaRAG. First, we describe the knowledge graph construction in Section 4.1. Second, based on the existing corpus, we outline the TeaRAG pipeline in Section 4.2. Finally, we introduce the training strategies employed to realize this pipeline in Section 4.3, including SFT and IP-DPO.

4.1 Knowledge Graph Construction

Most graph-enhanced RAG [42, 82] methods construct knowledge graphs from relatively small corpora (sub-million scale). Such settings cannot reliably evaluate model performance on large-scale data, as larger graphs inevitably introduce more noise and complexity. To investigate the performance of TeaRAG under a large-scale knowledge graph, we construct a knowledge graph based on the widely used Wikipedia corpus [31]. Following [91], we employ the Qwen2.5-14B-Instruct model [79] to extract knowledge triplets from each document chunk. This ensures that the content of the graph is consistent with the underlying corpus. The extracted entities are added to the entity set $\mathcal{V}$, and the extracted triplets are incorporated into the edge set $\mathcal{E}$. All entities and triplets obtained from the corpus together constitute the knowledge graph $\mathcal{G}$. Statistics for this graph are provided in Table 6.

The complete knowledge graph extraction from the Wikipedia corpus [31] requires approximately 389 GPU hours on A100 80G GPUs. This one-time preprocessing cost is effectively amortized over the system's deployment lifetime, as the constructed graph is persistently cached and reused across all subsequent queries. Furthermore, in online deployment settings, triplet extraction is required only for newly ingested documents. Since a single A100 80GB GPU processes approximately 15 chunks per second, the incremental extraction process is both efficient and cost-effective.

4.2 Pipeline of TeaRAG

Leveraging the Wikipedia corpus and its knowledge graph, the system can perform hybrid retrieval to autonomously address problems. In agentic RAG, conventional retrieval methods are confined to semantic

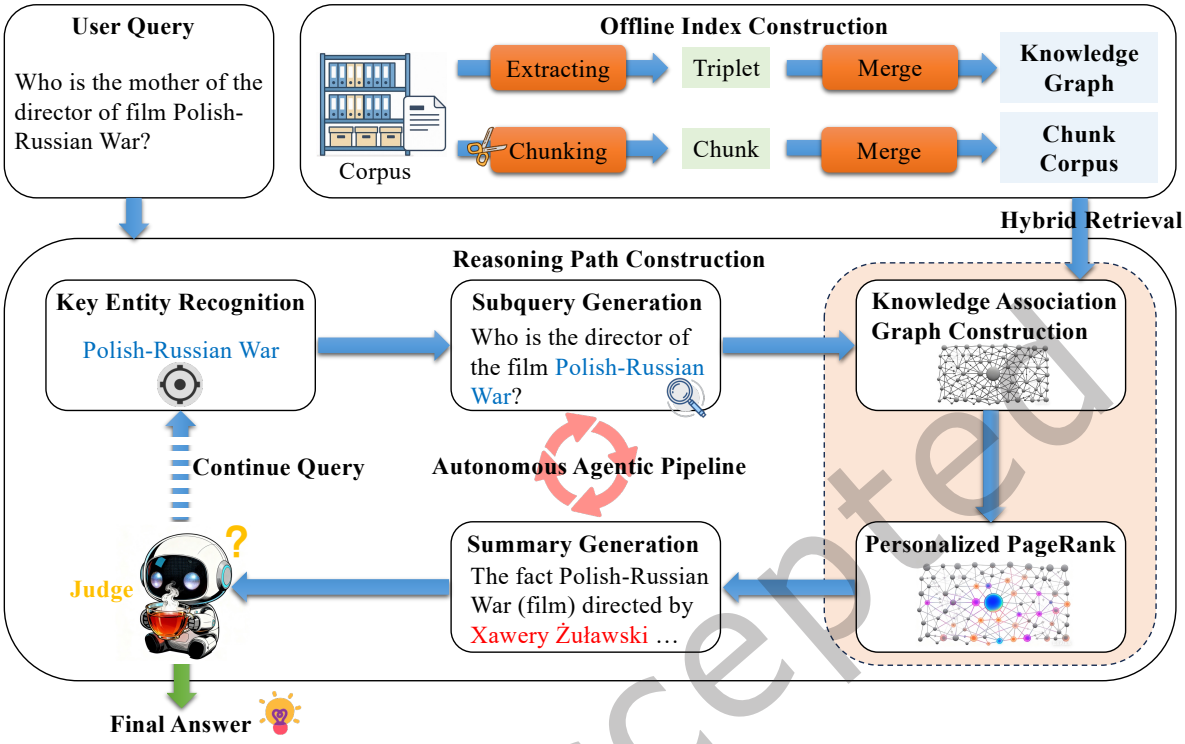


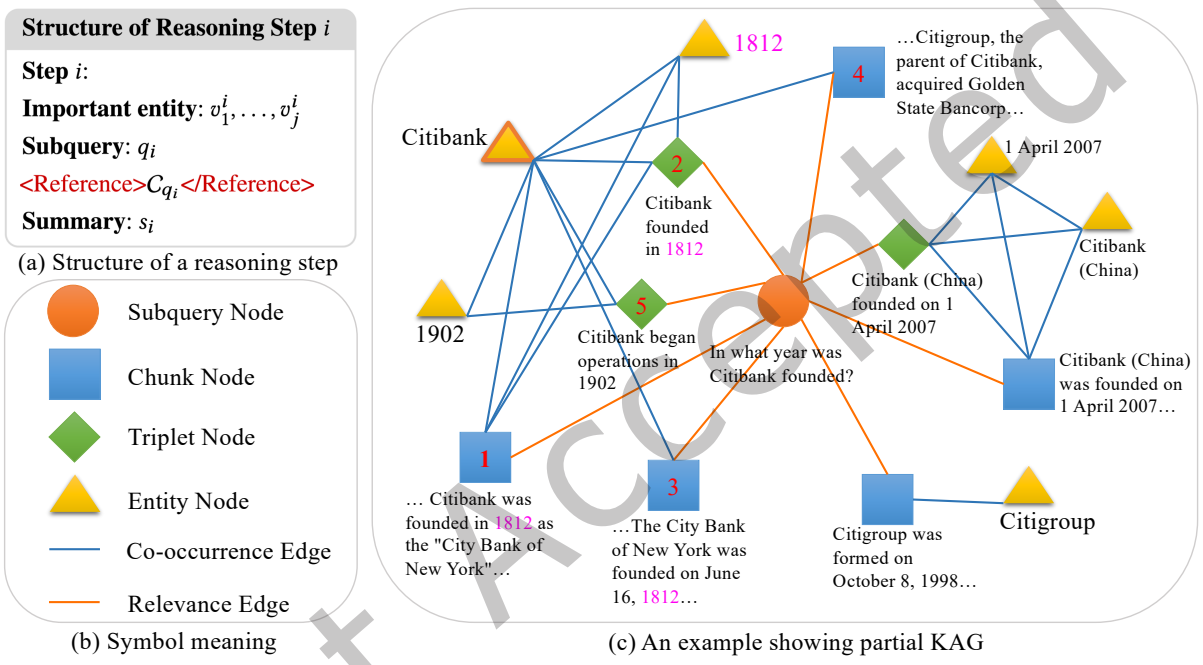
Fig. 3. The overall pipeline of TeaRAG. Based on an offline-built knowledge graph and chunk corpus index, TeaRAG progressively constructs a reasoning path until the final answer is determined.

retrieval [28, 59], graph retrieval [42, 82], or their naive concatenation [55, 72, 82], overlooking both their complementary strengths and the strong relevance signal from their co-occurrence. Semantic retrieval offers rich background and conceptual context but suffers from low information density, noise [2, 44, 78], and inefficient token use. Conversely, graph retrieval yields high-density triplets containing precise facts but lacking contextual grounding. Besides, when a chunk from semantic retrieval and a triplet from graph retrieval correspond to the same source data, their co-occurrence constitutes a robust relevance signal, functioning as a high-confidence filter. Therefore, TeaRAG exploits this intrinsic relationship between chunks and triplets to reconcile the accuracy–efficiency trade-off. By prioritizing co-occurring chunk–triplet pairs over less certain chunks, TeaRAG can enhance contextual accuracy while improving token efficiency for the LLM.

Consequently, we implement the pipeline of TeaRAG as shown in Fig. 3. We redesign the structure of the agent's reasoning path as $\mathcal{P}_k = [S_1, \ldots, S_k]$, where S_i denotes the i-th reasoning step. The structure of a reasoning step is shown in Fig. 4 (a). Specifically, after receiving the instruction prompt in Table 1, TeaRAG first identifies the key entities central to the current reasoning step. Based on these entities, it formulates the subquery required for the step. Subsequently, TeaRAG performs context retrieval using a hybrid approach, extracting relevant information from both the chunk corpus and the knowledge graph, and constructs a KAG to enable PPR filtering for precise and concise context. TeaRAG then summarizes the core content from this context with respect to the subquery. Finally, leveraging the previous reasoning steps, TeaRAG decides whether to proceed with further reasoning or generate the final answer. Next, we present the detailed design of a reasoning step.

Table 1. The instruction prompt utilized in TeaRAG. <Reference> and </Reference> are special tokens for wrapping retrieved information.

Follow these iterative steps to answer the overall question: 1. Important entity: Identify key entities to investigate for the current step. Multiple entities should be separated by ###. 2. Subquery: Formulate search queries based on important entities. 3. Search: Execute the search. All results MUST be placed within <Reference> and </Reference> tags. 4. Summary: Analyze the search results (e.g., based on clues, obtain sub-answers). 5. Decide: If the information is sufficient, provide the "Final Answer:". Otherwise, return to step 1.



(a) Structure of a reasoning step

(b) Symbol meaning

(c) An example showing partial KAG

Fig. 4. (a) shows the structure of a reasoning step. <Reference> and </Reference> are special tokens for wrapping retrieved information. (b) shows the meaning of symbol in (c). (c) shows an example of partial KAG. The answer is highlighted by pink text. The red number on nodes is the ranking of the content selected in PPR.

4.2.1 Important Entity Recognition. Before question decomposition and generation, LLMs need to identify the current anchor entity. By focusing on the key entities in the question, they can generate more targeted questions [48] and facilitate subsequent PPR filtering of irrelevant information. The key entities recognized by LLMs at step i are denoted as $v_1^i, \ldots, v_j^i$.

4.2.2 Subquery Generation. Based on current key entities and previous reasoning information, the LLM decomposes the original question, generating the current subquery q_i to be solved. The retrieval algorithms $\mathcal{A}_{\mathcal{R}}$ then acquire relevant context centered around this subquery q_i.

4.2.3 Context Retrieval. TeaRAG's context retrieval combines semantic and graph retrieval.

The first component is semantic retrieval. The retriever, $\mathcal{R}$, retrieves relevant chunks $\mathcal{D}_{q_i}$ from the document corpus $\mathcal{D}$ based on their similarity to the subquery q_i. This is denoted as: $\mathcal{D}_{q_i} = \mathcal{R}(q_i, \mathcal{D})$.

The second component is graph retrieval, which retrieves a set of knowledge triplets $\mathcal{E}_{q_i}$ from the knowledge graph $\mathcal{G}$. However, retrieving triplets based solely on identified key entities without considering detailed relationships risks collecting irrelevant triplets, whereas directly searching for relevant triplets from the entire set can be hampered by excessive noise from similar entries. To address these issues, we propose a two-stage graph retrieval method to perform graph retrieval in a more fine-grained manner. The first stage retrieves relevant entities from the entity set $\mathcal{V}$ using the retriever $\mathcal{R}$. For the set of key entities $\{v_1^i, \ldots, v_j^i\}$ from the current reasoning step i, we generate entity queries by pairing each entity v_t^i with the subquery q_i in the format: "Key entity: v_t^i. Query: q_i." The union of all entities returned by the retriever for these entity queries forms the set $\mathcal{V}_{q_i}$. We then collect all one-hop triplets connected to the entities in $\mathcal{V}_{q_i}$ from the knowledge graph, forming the set $\mathcal{E}_{entity}$. In the second stage, we retrieve the set of triplets most relevant to q_i from this constrained triplet set, defined as $\mathcal{E}_{q_i} = \mathcal{R}(q_i, \mathcal{E}_{entity})$.

In the above two components, we employ a top-K strategy to directly select the top-K contexts with the highest retrieval scores. We denote the size of these sets as $|\mathcal{D}_{q_i}| = k_d$ and $|\mathcal{E}_{q_i}| = k_t$. Next, our aim is to select the final k_f contexts for the LLM from the set $\mathcal{D}_{q_i} \cup \mathcal{E}_{q_i}$ by constructing a KAG based on the co-occurrence mechanism.

4.2.4 Knowledge Association Graph Construction. Based on the retrieved content, we construct a KAG, a heterogeneous graph designed to capture the co-occurrence relationships between different knowledge elements. This graph comprises four distinct node types and two types of association edges that connect them, as shown in Fig. 4 (b). The node types are defined as follows:

- **Subquery node n_{q_i}:** A single node representing the current subquery q_i. This node serves as the central anchor of the graph, and all other information is retrieved based on this.
- **Chunk node n_d:** Each node of this type represents a specific textual chunk d sourced from the retrieved document set $\mathcal{D}_{q_i}$.
- **Triplet node n_e:** Represents a knowledge triplet e from the triplet set $\mathcal{E}_{q_i}$.
- **Entity node n_v:** Each node of this type represents an entity v, which can be one of the entities $v_1^i, \ldots, v_j^i$ identified at reasoning step i, a source document title for a chunk, or a head or tail entity from a knowledge triplet.

The edges of the KAG capture either structural co-occurrence or semantic similarity, and are categorized as **co-occurrence edges** and **relevance edges**, respectively. Besides, all edges are undirected. Co-occurrence edges signify structural or contextual links between nodes. They are assigned a weight of 1 and are defined by the following rules:

- **Entity-Entity edge:** (n_{v^h}, n_{v^t}) connects the head and tail entities of a triplet $e = (v^h, rel, v^t)$.
- **Triplet-Entity edge:** (n_e, n_{v^h}) and (n_e, n_{v^t}) connect a triplet node to its constituent entities.
- **Chunk-Triplet edge:** If a triplet $e \in \mathcal{E}_{q_i}$ and its source chunk d is also in $\mathcal{D}_{q_i}$, an edge (n_e, n_d) is created, signifying a co-occurrence between the triplet and the chunk.
- **Chunk-Entity edge:** When a triplet (v^h, rel, v^t) co-occurs with a chunk d, edges are also added between the entity nodes of the triplet and the chunk node (n_d, n_{v^h}) and (n_d, n_{v^t}). Besides, an edge (n_d, n_{title}) connects a chunk node n_d to the entity node of its document title n_{title}.

Relevance edges quantify the relevance of chunk and triplet nodes to the subquery node, defined as:

- **Chunk-Subquery edge:** Weight of (n_d, n_{q_i}) for a chunk $d \in \mathcal{D}_{q_i}$ is based on their similarity:

$$w(n_d, n_{q_i}) = \text{sigmoid}(\text{sim}_{\mathcal{R}}(d, q_i)), \tag{1}$$

where $\text{sim}_{\mathcal{R}}$ is the relevance score from retriever $\mathcal{R}$, normalized via sigmoid.

- **Triplet-Subquery edge:** To account for the contextual sparsity of triplets, which can lead to unreliable similarity scores, we apply a thresholding mechanism. The weight of the edge (n_e, n_{q_i}) for a triplet $e \in \mathcal{E}_{q_i}$ is:

$$w(n_e, n_{q_i}) = \max(\text{sigmoid}(\text{sim}_{\mathcal{R}}(e, q_i)) - \tau, 0), \tag{2}$$

  where τ is a threshold hyperparameter.

4.2.5 Personalized PageRank Filtering. After constructing the KAG, we employ PPR to identify the most important content nodes. This process allows us to dynamically select key information and compress tokens by replacing text chunks with high-density triplets, while ensuring relevance. We denote the importance distribution across all nodes as π. The weighted adjacency matrix from the graph construction is normalized to produce the transition matrix W. Besides, we denote the personalization vector as p, which acts as a bias to steer the final importance distribution. To ensure the relevance of the context to the subquery, the values for the subquery node n_{q_i} in the personalized vector p are set to 1. Meanwhile, the key entities identified by the LLM have their corresponding values in p set to 0.5, serving as anchors for the question. Subsequently, we iterate for N rounds to obtain the updated importance distribution π:

$$\pi = \alpha W \pi + (1 - \alpha) p, \tag{3}$$

where α is a hyperparameter balancing the graph-derived importance with the personalization vector bias. Finally, based on the ranking of the scores in π, we select the k_f highest-scoring nodes from the chunk and triplet nodes to form the final context $\mathcal{C}_{q_i}$. This context is then wrapped in <Reference> and </Reference> tags to signal to the LLM that it is an external resource.

We show a KAG example in Fig. 4 (c). Core relevant knowledge forms dense graphs via co-occurrence edges. This indicates that using co-occurrence is usually a more precise way to filter noise and prevent the model's inference from being distracted by similar but irrelevant content.

4.2.6 Summary Generation. The LLM summarizes the collected external context to produce a summary, s_i, which enables subsequent steps to easily identify the key information for this step.

4.2.7 Final Answer Generation. The LLM autonomously determines whether the information collected in the reasoning path is sufficient. If not, it continues to generate the next reasoning step $S_{i+1} = \text{LLM}(I, q, \mathcal{P}_i)$. Conversely, when the information is sufficient, the LLM generates the final answer $a = \text{LLM}(I, q, \mathcal{P}_i)$. And the final answer a starts with the format "Final answer: ", which facilitates the format check.

4.3 Model Training

To enable LLMs to autonomously execute the aforementioned pipeline, current advanced agentic RAG mainly adopts outcome-based RL [28, 59]. However, these methods rely on high-resource RL methods such as PPO [56] and GRPO [57], which require continuous inference and retrieval during training, and these methods only focus on the final results, making it difficult to effectively control the reasoning steps. Although some methods utilize GPT-4o to annotate intermediate reasoning steps [88] or consider complex rewards in the PPO framework [71], these approaches make it difficult to achieve efficient, low-resource training of agentic RAG. Therefore, to control the number of reasoning steps k in $\mathcal{P}_k$ and enable efficient training, we propose a two-stage training paradigm that improves the efficiency and conciseness of reasoning by introducing process-level rewards. The overall training framework is shown in Fig. 5. In the first stage, SFT trains the model on basic reasoning patterns. In the second stage, IP-DPO is applied to further enhance the model's generalization.

4.3.1 First Stage: SFT. To build reasoning SFT data, we use the Musique dataset [63], which provides structured query decomposition and supporting golden evidence paragraphs. Each reasoning step is constructed as follows:

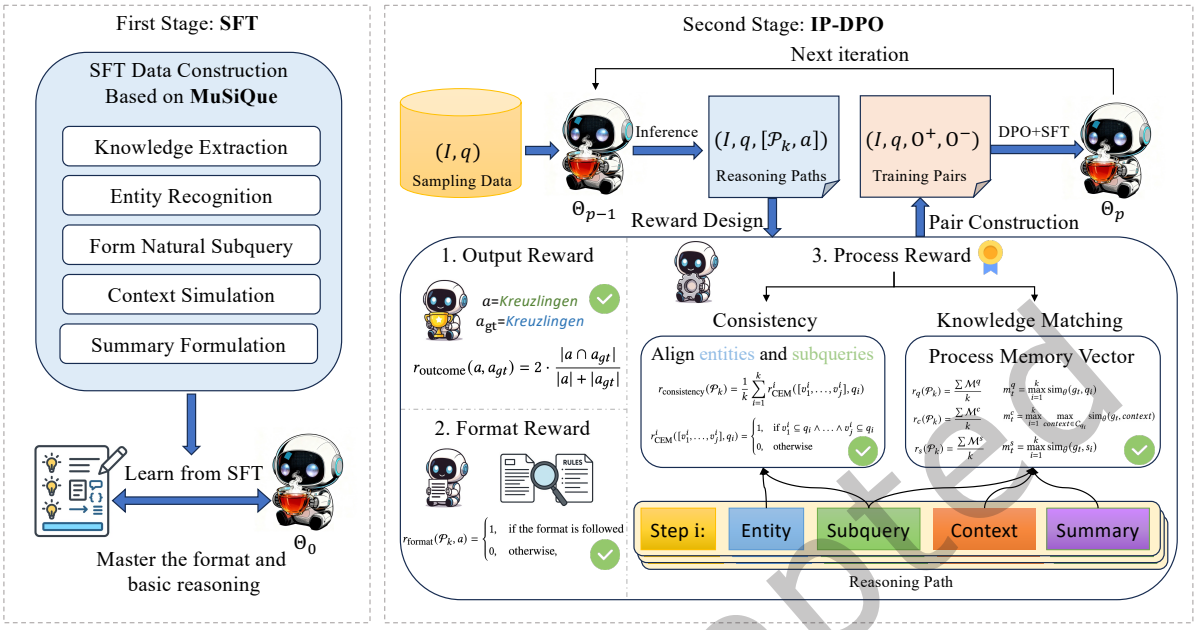


Fig. 5. The overall training framework for TeaRAG follows a two-stage paradigm. First, we conduct SFT on the preprocessed MuSiQue dataset to help the LLM learn the required format and develop basic reasoning skills. In the second stage, we apply IP-DPO with a process-aware reward to further improve the model while preventing overthinking.

(1) **Knowledge extraction:** We first process the golden evidence paragraphs of the current subquery with Qwen2.5-14B-Instruct to extract a set of knowledge triples, which are aligned with our knowledge graph construction. From this set, Qwen2.5-72B-Instruct then identifies the supporting triplets that directly answer each intermediate subquery.
(2) **Important entity recognition:** We view the entities in supporting triplets as anchor entities.
(3) **Subquery generation:** We employ Qwen2.5-72B-Instruct to transform the structured intermediate questions from Musique into fluent, natural language subqueries.
(4) **Context simulation:** To improve fact localization, we synthetically construct a context for training. We create this context by taking the golden evidence paragraphs and randomly inserting the evidence triplets. This process helps the LLM learn to adapt to diverse and imperfect contexts.
(5) **Summary formulation:** We use diverse templates to construct target summaries. Every summary follows a two-part structure. It first presents the key supporting triplets and then concludes with the answer to the subquery.

After constructing each reasoning step, we concatenate them in the order specified by the Musique dataset to form a complete reasoning path. Alternatively, each individual reasoning step can be treated as a single-hop question. We denote the SFT dataset as $\mathcal{S}_{SFT} = \{(I, q, O)\} = \{(I, q, [\mathcal{P}_k, a])\}$, where O represents either the direct output of the LLM or context autonomously retrieved by the LLM. The SFT dataset statistics are shown in Table 2.

To avoid interfering with the LLM's ability to learn reasoning patterns due to the retrieved context, following [28], we mask the context between the two special tokens <Reference> and </Reference>. We fine-tune the

model on the SFT dataset, using the following objective function:

$$\mathcal{L}_{SFT} = -\mathbb{E}_{(I,q,O)\sim\mathcal{S}_{SFT}} \sum_{t=1}^{|O|} \mathbb{I}[O_t \text{ not masked}] \log P(O_t \mid I, q, O_{<t}; \Theta), \tag{4}$$

where $\mathbb{I}$ is the indicator function, and Θ denotes the parameters of the LLM. Through this SFT training phase, the model can generate outputs following the constructed reasoning path structure and acquire preliminary reasoning capabilities.

4.3.2 Second Stage: IP-DPO. The models trained with SFT are not optimized for real-world retrieval environments, where evidence is often noisy and incomplete. Consequently, their reasoning abilities are poorly suited to realistic retrieval scenarios, and they exhibit limited generalization [33]. To address this issue and control the number of reasoning steps k, we propose IP-DPO, which introduces a process-aware reward and uses iterative DPO [54] to optimize the LLMs for better and more concise reasoning paths. First, we collect a sampling dataset containing a diverse set of tasks and use the LLM to generate outputs for this dataset. Next, we score each sampled instance with a process-aware reward and the final outcome. These scored data are then used to construct data pairs, which serve as the training inputs for IP-DPO. After each training cycle, we resample using the updated model and repeat this entire process iteratively to improve performance further.

Dataset collection. To enhance data diversity, we collect a dataset by sampling from the training sets of NQ [34], HotpotQA [80], and Musique [63]. From the NQ dataset, we randomly sample 4,000 single-hop questions. From the HotpotQA dataset, we sample questions from the hard level that contain as many evidence sentences as possible, drawing 2,500 questions each from the bridge and comparison types. From the Musique dataset, we sample a total of 1,000 questions, distributed evenly across the 2-hop, 3-hop, and 4-hop data. The data statistic is shown in Table 3. The model then performs inference using the overall pipeline on each question in this dataset to repeatedly generate R reasoning paths. For each reasoning path, both the reasoning process and the final answer are recorded for subsequent evaluation.

Reward design. To better guide the LLM's reasoning process and encourage more concise reasoning steps, we design a process-aware reward based on knowledge matching. This enables a more fine-grained evaluation of the quality of different generated outputs. Our total reward system consists of three components: the outcome reward, the format reward, and the process reward.

The first part is the outcome reward, which is based on a comparison between the final answer generated by the LLM and the ground truth. Because rewards based on an exact match are too sparse to effectively compare the accuracy of LLM-generated answers, we follow [59] and use the F1 score as the metric for the outcome reward. Specifically, we define the outcome reward as follows:

$$r_{\text{outcome}}(a, a_{gt}) = 2 \cdot \frac{|a \cap a_{gt}|}{|a| + |a_{gt}|}, \tag{5}$$

where a_{gt} represents the ground truth, $|a|$ is the word count of the LLM-generated answer a, and $|a \cap a_{gt}|$ is the number of overlapping words between a and a_{gt}.

The second part is the format reward, which evaluates whether the structure of the model's reasoning path conforms to the structure constructed in Section 4.2. The format reward of reasoning path $\mathcal{P}_k$ and final answer a is defined as follows:

$$r_{\text{format}}(\mathcal{P}_k, a) = \begin{cases} 1, & \text{if the format is followed} \\ 0, & \text{otherwise,} \end{cases} \tag{6}$$

The third part is the process reward, which assesses the rationality of the LLM's reasoning process. The process rewards include two components: entity-subquery consistency reward and knowledge matching rewards. The

first component is the entity-subquery consistency reward, intended to align the entities identified by the LLM with the anchor of the subsequent subquery. This alignment enables more targeted question decomposition and more precise PPR filtering. We implement a cover exact match (CEM) mechanism that mandates complete inclusion of key entities within the subquery. The corresponding reward function is defined as follows:

$$r_{\text{consistency}}(\mathcal{P}_k) = \frac{1}{k}\sum_{i=1}^{k} r^i_{\text{CEM}}([v^i_1, \dots, v^i_j], q_i), \tag{7}$$

$$r^i_{\text{CEM}}([v^i_1, \dots, v^i_j], q_i) = \begin{cases} 1, & \text{if } v^i_1 \subseteq q_i \wedge \dots \wedge v^i_j \subseteq q_i \\ 0, & \text{otherwise} \end{cases}. \tag{8}$$

The second component is the knowledge matching rewards. These rewards evaluate the correctness of the LLM's reasoning process by assessing whether its intermediate steps successfully capture the golden evidence. We evaluate three types of intermediate steps: subqueries, retrieved contexts, and summaries. We denote the set of golden evidence for an overall question as $G = [g_1, \dots, g_l]$, where l is the number of golden evidence pieces. The non-deterministic nature of the reasoning process makes it difficult to pre-assign which specific step should capture a particular piece of golden evidence. To address this, following [71], we create a memory vector for each type of intermediate step. This vector stores the maximum similarity score achieved between each piece of evidence and any step in the reasoning path $\mathcal{P}_k$. This score serves as a metric to evaluate whether the model successfully integrated the required evidence. Specifically, we denote the subquery memory vector as $\mathcal{M}^q = [m^q_1, \dots, m^q_l]$. Here, m^q_t records the maximum similarity between the golden evidence g_t and any subquery q_i in the reasoning path (where $1 \le t \le l$):

$$m^q_t = \max_{i=1}^{k} \text{sim}_\theta(g_t, q_i), \tag{9}$$

where $\text{sim}_\theta()$ is a similarity function based on a reranker model θ, which uses a sigmoid function to normalize the score to the range $[0, 1]$. Similarly, we define the context memory vector $\mathcal{M}^c$ and the summary memory vector $\mathcal{M}^s$. The memory elements, m^c_t and m^s_t, are calculated as follows:

$$m^c_t = \max_{i=1}^{k} \max_{context \in \mathcal{C}_{q_i}} \text{sim}_\theta(g_t, context), \tag{10}$$

$$m^s_t = \max_{i=1}^{k} \text{sim}_\theta(g_t, s_i). \tag{11}$$

These calculations yield three memory vectors, $\mathcal{M}^q$, $\mathcal{M}^c$, and $\mathcal{M}^s$, for the LLM's reasoning process. For a multi-hop question with l pieces of golden evidence, an ideal path might decompose the problem into approximately l subqueries. For a reasoning path $\mathcal{P}_k$ with k steps, a value of $k \le l$ suggests a concise reasoning process without redundant retrievals. Conversely, when $k > l$, it may indicate inefficiencies such as overthinking or repetitive searches. To promote conciseness and penalize inefficiency, we normalize the summed memory scores by the number of steps, k. The final rewards are calculated as follows:

$$r_q(\mathcal{P}_k) = \frac{\sum \mathcal{M}^q}{k}, \quad r_c(\mathcal{P}_k) = \frac{\sum \mathcal{M}^c}{k}, \quad r_s(\mathcal{P}_k) = \frac{\sum \mathcal{M}^s}{k}, \tag{12}$$

where $r_q(\mathcal{P}_k)$, $r_c(\mathcal{P}_k)$, and $r_s(\mathcal{P}_k)$ are the process rewards for subquery generation, the retrieved context, and the summary, respectively. Based on these, the overall process reward is as follows:

$$r_{process}(\mathcal{P}_k) = 0.1 r_{\text{consistency}}(\mathcal{P}_k) + 0.3 r_q(\mathcal{P}_k) + 0.3 r_c(\mathcal{P}_k) + 0.3 r_s(\mathcal{P}_k). \tag{13}$$

DPO pair construction. Based on the above reward design, each reasoning path can obtain three rewards: r_{format}, $r_{outcome}$, and $r_{process}$. The core challenge in DPO [54] is constructing positive-negative preference pairs. To this end, we design the following pair construction algorithm.

First, we select the chosen response set, which exhibits a more accurate reasoning process. Our approach varies based on the question type. For single-hop questions, which are typically simple and whose answers can be found with a single retrieval, we do not apply a process reward. This is also because our single-hop dataset lacks golden evidence. Instead, we directly scale the outcome reward by dividing it by the length of the reasoning path k, setting $r_{outcome} = \frac{r_{outcome}}{k}$ and $r_{process} = 0$. This approach incentivizes the LLM to solve single-hop questions in a single step. For more complex multi-hop questions, we utilize the process reward to differentiate and evaluate the quality of the reasoning steps. When considering format and outcome rewards, we must also ensure that the process reward is sufficiently high. Furthermore, many reasoning paths in multi-hop questions demonstrate relatively good reasoning processes, but due to variations in the presentation of the output, there are cases where small differences exist between the outcome and ground truth. Therefore, we have relaxed the conditions. Given a question q, the chosen response set $\mathcal{W}$ is formally defined as:

$$\mathrm{W} = \left\{ (q, \mathcal{P}_k, a, r_{format}, r_{outcome}, r_{process}) \;\middle|\; \begin{cases} r_{format} = 1 \wedge r_{outcome} = 1, & \text{if } q \text{ is single-hop} \\ r_{format} = 1 \wedge \Big((r_{outcome} = 1 \wedge r_{process} \geq 0.7) \vee \\ \quad (r_{outcome} \geq 0.8 \wedge r_{process} \geq 0.8) \Big), & \text{if } q \text{ is multi-hop} \end{cases} \right\}. \tag{14}$$

Next, for each selected response in $\mathcal{W}$, we choose a corresponding rejected response. Given a chosen response $(q, \mathcal{P}_k^+, a^+, r^+_{\text{format}}, r^+_{\text{outcome}}, r^+_{\text{process}})$, we rank sampled reasoning paths for the question q in ascending order based on the sum of the three reward scores. While traversing this ranked list sequentially, we define three types of rejected samples:

- **Format rejection:** $r^-_{\text{format}} = 0$. The reasoning path violates the required format.
- **Easy rejection:** $r^-_{\text{format}} = 1$ and $r^-_{\text{outcome}} \leq 0.3$. The generated reasoning path adheres to the format but fails to reach the correct answer.
- **Hard rejection:** $r^-_{\text{format}} = 1$, $0.3 < r^-_{\text{outcome}} \leq r^+_{\text{outcome}} - 0.3$, and $r^-_{\text{process}} \leq r^+_{\text{process}}$. The path largely progresses toward an answer but still contains specific errors.

To maintain the data diversity of DPO pairs, once a rejected response is obtained for a chosen sample, the algorithm continues traversing the ranked list from the next position to find the next rejected sample for the next chosen sample of the question q. This approach prevents the selection of duplicate rejected samples. Additionally, for single-hop questions, due to their simplicity, rejected samples are limited to format and easy rejection. Following this procedure, we construct the DPO pair dataset. We denote the dataset at iteration p by $\mathcal{S}_{DPO-p} = \{(I, q, O^+, O^-)\}$.

Iterative DPO training. Given that the quality of samples generated by the preliminary SFT model remains suboptimal, the performance gains from a single round of DPO training are inherently limited. To address this, we adopt an iterative DPO procedure designed to enhance the model's comprehension of preference patterns and enable progressively stronger alignment. Following [51, 52], we jointly optimize the DPO objective together with the SFT objective to mitigate distribution shift in the model's chosen responses. The training losses are defined as follows:

$$\mathcal{L}_{SFT} = -\mathbb{E}_{(I,q,O^+,O^-)\sim\mathcal{S}_{DPO-p}} \sum_{t=1}^{|O^+|} \mathbb{I}[O_t^+ \text{ not masked}] \log P(O_t^+ \mid I, q, O^+_{<t}; \Theta_p), \tag{15}$$

$$\begin{aligned} \mathcal{L}_{DPO} = -\mathbb{E}_{(I,q,O^+,O^-)\sim\mathcal{S}_{DPO-p}} \Bigg[\log \sigma \Bigg(& \beta \sum_{t=1}^{|O^+|} \mathbb{I}[O_t^+ \text{ not masked}] \log \frac{P(O_t^+ \mid I, q, O^+_{<t}; \Theta_p)}{P(O_t^+ \mid I, q, O^+_{<t}; \Theta_{p-1})} \\ & -\beta \sum_{t=1}^{|O^-|} \mathbb{I}[O_t^- \text{ not masked}] \log \frac{P(O_t^- \mid I, q, O^-_{<t}; \Theta_p)}{P(O_t^- \mid I, q, O^-_{<t}; \Theta_{p-1})} \Bigg) \Bigg], \end{aligned} \tag{16}$$

$$\mathcal{L}_{ALL} = \eta \mathcal{L}_{SFT} + \mathcal{L}_{DPO}, \tag{17}$$

where β and η are hyperparameters, and Θ_p denotes the model parameters in the p-th round, initialized from Θ_{p-1}. Θ_0 represents the model parameters after the first-stage SFT.

Once the model completes training with the aforementioned loss function, the next round of data sampling, sample reward scoring, pair construction, and training can be performed, implementing iterative DPO training to continuously optimize the model's capabilities.

5 Experiments

5.1 Experiment Setup

5.1.1 Datasets and Evaluation Metrics. We use the training datasets from Section 4.3 to perform SFT and IP-DPO. The statistics of these datasets are shown in Table 2 and Table 3. Besides, we focus on question-answer (QA) datasets. For single-hop QA datasets, we use **NQ** [34] and **PopQA** [49]. For multi-hop QA datasets, we use **HotpotQA** [80], **2WikiMultiHopQA** [19], **Musique** [63], and **Bamboogle** [53]. For datasets without a test set, we use the development set for testing. The statistical details of these datasets are shown in Table 4. Since we use some training data from NQ, HotpotQA, and Musique, the tests on NQ, HotpotQA, and Musique datasets are in-domain tests, and the tests on PopQA, 2WikiMultiHopQA, and Bamboogle are out-of-domain tests. For the retrieval corpus, following [29], we adopt the Wikipedia corpus [31], as the chunk corpus. The statistics of the chunk corpus are shown in Table 5. Based on the knowledge graph construction described in Section 4.1, we constructed a large-scale knowledge graph, and the statistical information of this knowledge graph is shown in Table 6. For all datasets, we adopt EM and F1 as the evaluation metrics.

5.1.2 Baselines. We compare our TeaRAG with various baselines, which are divided into the following two categories: one-turn generation and iterative RAG.

One-turn generation prepares all the required information and utilize the LLM in a single pass to generate the final answer. The methods in this category are as follows:

- **Zero-shot:** This method uses the LLM to generate answers without retrieving relevant information.
- **RAG** [36]**:** This is the standard RAG method that uses semantic retrieval to search for relevant text chunks from the corpus, which are then input as evidence to the LLM to directly generate answers.
- **Rerank** [84]**:** This method uses semantic retrieval to find relevant chunks, then applies reranking to filter noise, before inputting them as evidence to the LLM to directly generate answers.
- **RECOMP**$_{abs}$ [78]**:** This method uses a compressor to extract core information related to the question from retrieved chunks before inputting them to the LLM.
- **MixPR-RAG** [1]**:** This method constructs a semantic graph from each sentence in the chunks and uses PPR to filter out important sentences to input to the LLM.

Iterative RAG utilizes LLMs to autonomously determine whether retrieval is needed, making multiple calls to retrieval tools and integrating multiple retrieval contexts to derive the final answer. The methods in this category are as follows:

- **IRCoT** [64]**:** This method iteratively utilizes Chain-of-Thought (CoT) to guide retrieval and uses retrieval results to enhance CoT to derive the final answer.
- **SelfRAG** [2]**:** This method utilizes LLMs to generate special tokens to help the model decide when to retrieve and whether to use the retrieved content.
- **R1-Searcher** [59]**:** This method employs two-stage training. The first stage trains the model to learn generating retrieval queries, and the second stage uses outcome-based and format rewards to train LLMs to automatically invoke retrieval during reasoning. Based on Reinforce++ [21], this method is trained using data from HotpotQA

Table 2. SFT data distribution.

Dataset	1-hop	2-hop	3-hop	4-hop	Total
Train	2,700	1,800	1,800	1,057	7,357
Test	300	200	200	118	818

Table 3. IP-DPO dataset composition.

Source Dataset	Number
NQ	4,000
HotpotQA	5,000
Musique	1,000
Total	10,000

Table 4. QA datasets for testing.

Task	Dataset	# Dev	# Test
Single-hop QA	NQ	8,757	3,610
	PopQA	–	14,267
Multi-hop QA	HotpotQA	7,405	–
	2WikiMultiHopQA	12,576	–
	Musique	2,417	–
	Bamboogle	–	125

Table 5. Chunk corpus statistics.

Metric	Value
Size of document set	3,232,908
Size of chunk set $\mathcal{D}$	21,015,324
Average length of chunk	100

Table 6. Knowledge graph statistics.

Metric	Value
Size of entity set $\mathcal{V}$	51,063,765
Size of relation set $\mathcal{E}$	130,931,564
Average out-degree per head entity	9.24
Average in-degree per tail entity	3.05
Average degree per entity	5.13

and 2WikiMultiHopQA. We use two versions of models trained by this method: R1-Searcher-Llama-8B and R1-Searcher-Qwen-7B.

- **Search-R1** [28]**:** This method utilizes PPO [56] with complete NQ and HotpotQA training data to optimize LLMs based on outcome rewards. We test four versions of models trained by this method: Search-R1-instruct-7B, Search-R1-base-7B, Search-R1-instruct-14B, Search-R1-base-14B.

For all baselines, the retrieval systems use E5-base-V2 [69] as the retriever unless otherwise specified. The "**+R**" symbol indicates that the retrieval system incorporates both retrieval and reranking stages, with BGE-reranker-v2 [5] serving as the reranker model. This configuration ensures a fair comparison with our TeaRAG.

5.1.3 Implementation Details. To ensure robustness, we employ two widely used LLMs for training and inference: Llama3-8B-Instruct [14] and Qwen2.5-14B-Instruct [79]. For the retrieval stage, following [29], we utilize E5-base-V2 [69] as the retriever and BGE-reranker-v2 [5] as the reranker, which together constitute our retrieval system $\mathcal{R}$. For semantic retrieval, we retrieve the top-20 chunks, which are then reranked to a final set of $k_d = 5$ chunks. For graph retrieval, we initially retrieve the top-10 entities and 20 edges. After reranking, these are narrowed down to 5 entities and $k_t = 10$ edges. The final number of items composing the context is set to $k_f = 5$. During the

Table 7. Performance comparison of various methods on six QA benchmarks. "*" indicates testing using officially released LLMs. "+**R**" indicates that the retrieval system includes retrieval and rerank stages. The best results are highlighted in bold. "†" indicates the statistically significant improvements in average results (i.e., two-sided t-test with $p < 0.05$) over all baselines.

LLM	Method	NQ		PopQA		HotpotQA		2Wiki		Musique		Bamboogle		Avg.	
		EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Llama3-8B	**zero-shot**	22.49	31.62	22.56	26.84	19.14	26.91	17.81	26.14	4.30	10.00	12.80	18.91	16.41	23.40
	RAG	36.03	47.19	37.73	46.43	25.29	35.69	8.19	20.18	4.71	10.26	8.80	17.82	20.12	29.59
	Rerank	37.28	48.48	41.08	50.27	29.53	40.80	9.64	22.12	5.54	12.08	11.20	19.94	22.37	32.28
	RECOMP$_{abs}$	35.76	45.72	42.79	49.08	27.77	37.71	23.17	29.86	5.00	10.56	10.40	19.69	24.14	32.10
	MixPR-RAG	26.73	38.88	35.22	44.96	23.07	34.47	7.96	21.02	3.72	10.08	8.80	18.85	17.58	28.04
	IRCoT	35.31	46.57	39.72	47.42	33.50	44.67	25.81	34.58	9.43	15.87	32.80	43.21	29.42	38.72
	SelfRAG*	33.90	41.86	19.21	31.27	16.12	28.36	11.72	23.24	4.38	12.49	4.00	14.47	14.88	25.28
	R1-Searcher-Llama-8B*	39.92	49.89	42.01	47.31	43.46	55.12	44.92	50.12	17.54	25.46	44.00	55.57	38.64	47.25
	R1-Searcher-Llama-8B*+R	39.92	50.27	44.31	49.79	45.77	57.73	46.39	51.41	18.58	26.53	40.80	54.10	39.29	48.30
	R1-Searcher-Qwen-7B*	40.33	50.91	39.03	45.33	45.01	57.51	47.64	53.91	21.97	30.86	45.60	55.39	39.93	48.98
	R1-Searcher-Qwen-7B*+R	42.08	52.95	41.48	47.94	47.56	**60.40**	**49.16**	**55.55**	22.59	31.80	40.80	55.46	40.61	50.68
	Search-R1-instruct-7B*	40.25	50.41	40.32	45.88	38.47	49.23	35.67	41.88	16.71	24.53	41.60	53.33	35.50	44.21
	Search-R1-base-7B*	49.11	57.86	47.14	51.23	44.89	56.81	37.02	43.03	20.60	29.28	48.80	59.73	41.26	49.66
	Search-R1-base-7B*+R	**50.39**	59.09	49.86	54.00	**47.66**	60.14	39.49	45.65	21.64	30.57	**51.20**	**64.51**	43.37	52.33
	TeaRAG-SFT-8B	38.56	47.46	46.56	50.67	28.86	38.23	26.39	31.99	18.99	27.57	36.00	47.96	32.56	40.65
	TeaRAG-8B	50.06	**59.71**	**51.98**	**56.08**	46.59	59.48	47.89	54.21	**26.98**	**37.34**	47.20	59.89	**45.12**†	**54.45**†
Qwen2.5-14B	**zero-shot**	21.57	31.16	20.40	24.42	22.61	31.38	26.78	32.58	5.13	13.79	15.20	25.06	18.61	26.39
	RAG	32.77	46.03	37.97	46.51	28.14	39.55	18.94	29.13	6.04	12.76	16.00	29.81	23.31	33.96
	Rerank	33.29	46.71	40.87	49.75	32.47	44.80	21.66	31.88	7.86	15.77	22.40	32.03	26.42	36.82
	RECOMP$_{abs}$	34.87	45.25	43.64	49.42	27.42	38.25	23.17	30.54	5.33	11.18	14.40	23.55	24.80	33.03
	MixPR-RAG	33.40	46.42	38.57	46.76	31.51	43.73	21.77	32.16	6.74	14.74	18.40	29.87	25.06	35.61
	IRCoT	28.31	39.95	34.25	41.43	23.76	34.43	10.95	19.29	7.36	12.12	24.00	34.67	21.43	30.31
	SelfRAG*	33.90	41.86	19.21	31.27	16.12	28.36	11.72	23.24	4.38	12.49	4.00	14.47	14.88	25.28
	R1-Searcher-Llama-8B*	39.92	49.89	42.01	47.31	43.46	55.12	44.92	50.12	17.54	25.46	44.00	55.57	38.64	47.25
	R1-Searcher-Llama-8B*+R	39.92	50.27	44.31	49.79	45.77	57.73	46.39	51.41	18.58	26.53	40.80	54.10	39.29	48.30
	R1-Searcher-Qwen-7B*	40.33	50.91	39.03	45.33	45.01	57.51	47.64	53.91	21.97	30.86	45.60	55.39	39.93	48.98
	R1-Searcher-Qwen-7B*+R	42.08	52.95	41.48	47.94	47.56	60.40	49.16	55.55	22.59	31.80	40.80	55.46	40.61	50.68
	Search-R1-instruct-14B*	43.24	53.49	45.37	51.24	45.55	56.96	42.69	48.15	21.68	29.70	48.00	60.20	41.09	49.96
	Search-R1-base-14B*	49.67	58.36	48.10	51.69	47.78	59.97	46.33	52.22	24.95	33.40	**51.20**	64.78	44.67	53.40
	Search-R1-base-14B*+R	**51.27**	60.07	50.33	53.89	**49.93**	62.31	48.88	54.51	27.18	35.46	**51.20**	**65.62**	46.47	55.31
	TeaRAG-SFT-14B	43.52	52.77	49.60	53.88	39.42	50.40	36.34	42.39	21.56	32.23	40.00	52.79	38.41	47.41
	TeaRAG-14B	50.33	**60.31**	**53.59**	**58.02**	**49.93**	**63.14**	**52.29**	**57.91**	**27.93**	**39.22**	50.40	64.07	**47.41**†	**57.11**†

PPR process, we set the hyperparameters $\tau = 0.2$ and $\alpha = 0.5$. The iteration N of PPR is set to 200. We configure the LLM to perform a maximum of 5 reasoning steps, i.e., $k \leq 5$.

Our experiments are conducted using 8 NVIDIA A100 (80G) GPUs. In the SFT phase, the learning rate is 5×10^{-4}, with an overall batch size of 128 and a per-device batch size of 16. We train for 1 epoch with a weight decay of 0.001. In the IP-DPO phase, $R = 8$ reasoning paths are repeatedly sampled for each question. And the number of epochs for the first two DPO rounds is 2, while the final round is trained for only 1 epoch to prevent overfitting from data repetition. The hyperparameter β is set to 0.5. The SFT weighting parameter η is tuned within the range $[0.25, 0.5, 1]$ and is progressively reduced as DPO iterations increase to prevent overfitting. Besides, the learning rate is set to 1×10^{-4}, the overall batch size is 16, the per-device batch size is 2, and the weight decay is 0.001. In both training stages, we adopt LoRA [20] for parameter-efficient fine-tuning. We set the LoRA rank to 8, α to 32, and dropout to 0.1, and apply it to the q_proj, k_proj, v_proj, and o_proj layers.

5.2 Overall Performance

The overall performance of the model is shown in Table 7. We have the following observations:

(1) **Our TeaRAG outperforms existing agentic RAG methods on most datasets and achieves the best average performance across all datasets.** This empirical result demonstrates the effectiveness of TeaRAG, which successfully boosts the performance of LLMs on RAG tasks by constructing a KAG-based agentic workflow combined with process supervision, outperforming previous outcome-based agentic RAG approaches.
(2) **The two-stage training paradigm significantly boosts the performance of TeaRAG.** In the first training stage, SFT yields substantial performance improvements for TeaRAG-SFT, surpassing both the prompt-based agentic RAG approach, IRCOT, and the SFT-trained agentic RAG system, SelfRAG. Based on TeaRAG-SFT, the second stage, IP-DPO, delivers a pronounced enhancement. Specifically, IP-DPO achieves a relative gain in average EM score of 38.57% on Llama3-8B-Instruct and 23.43% on Qwen2.5-14B-Instruct. These improvements highlight the effectiveness of IP-DPO in strengthening the model's generalization capability.
(3) **TeaRAG demonstrates strong out-of-domain generalizability.** Across both single-hop and multi-hop out-of-domain datasets, including PopQA, 2WikiMultiHopQA, and Bamboogle, our method delivers excellent performance. On the 2WikiMultiHopQA dataset, using the Llama3-8B-Instruct base model, TeaRAG-8B not only surpasses Search-R1-base-7B+R in the out-of-domain test by a significant margin but also achieves results on par with those of R1-Searcher-Qwen-7B+R in the in-domain evaluation.
(4) **More powerful LLMs achieve superior results.** This shows that TeaRAG leverages the core capabilities of LLMs. As the LLM's power increases, the agentic RAG's information-gathering abilities are correspondingly enhanced.
(5) **More advanced retrieval systems yield better results.** For a fair comparison with TeaRAG, we enhanced the original Search-R1 and R1-Searcher with a reranking-integrated retrieval system, which led to a significant performance increase. The continued superiority of TeaRAG over these augmented baselines highlights the effectiveness of its context retrieval strategy.

5.3 Analysis of Reasoning Paths

In this section, we analyze the reasoning paths of test instances across all datasets, investigating the distribution of the number of reasoning steps and the token usage of these reasoning paths.

5.3.1 Distribution of the Number of Reasoning Steps. We present the detailed reasoning step distribution of TeaRAG and other similarly-scaled agentic RAG baselines in Fig. 6. Since most methods require at least one reasoning step, we show the distribution of reasoning steps ranging from 1 to 5. We obtain the following observations:

(1) **TeaRAG requires shorter reasoning steps compared to agentic RAG baseline methods.** This indicates that by incorporating process supervision signals, we effectively guide the LLM's reasoning process, reducing indirection and improving overall efficiency.
(2) **Stronger information retrieval capabilities slightly improve reasoning efficiency.** For instance, by enhancing retrieval capabilities in the reranking stage, Search-R1-base-14B+R shows an increase in the proportion of reasoning completed in one or two steps to 54.3%, up from 50.9% for Search-R1-base-14B. This finding indirectly validates the effectiveness of TeaRAG's retrieval strategy, which leverages KAG construction to improve information accuracy and prevent repetitive searches that occur when critical information is not retrieved.
(3) **The distribution of reasoning path lengths for TeaRAG is stable across different base models.** In contrast, for the Search-R1 baseline, the proportion of one-step reasoning drops from 22.0% with Search-R1-base-7B+R to just 3.4% with Search-R1-base-14B+R. This suggests that the Search-R1 algorithm is highly

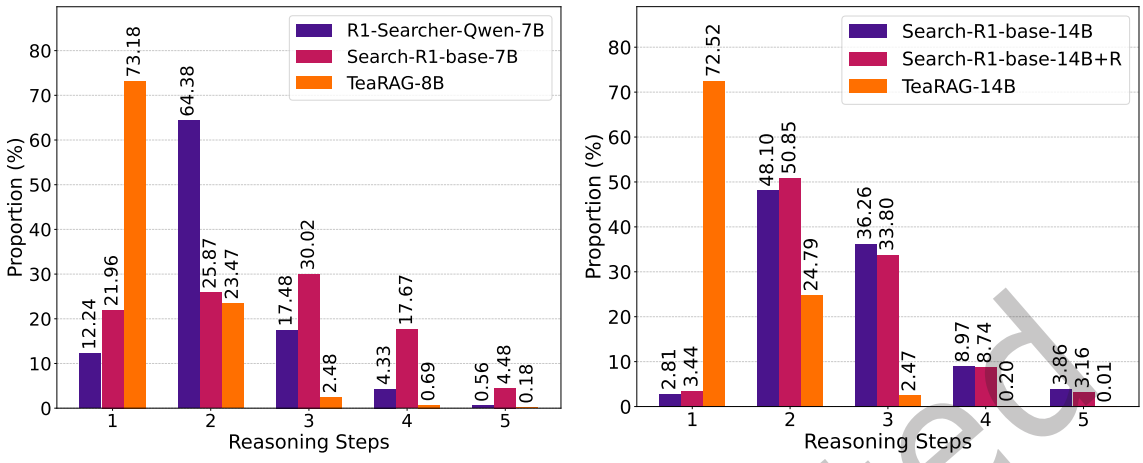


Fig. 6. Comparison of reasoning step distributions between TeaRAG and other baselines on six QA benchmarks. The figure on the left depicts the comparative results for TeaRAG-8B, while the figure on the right depicts those for TeaRAG-14B.

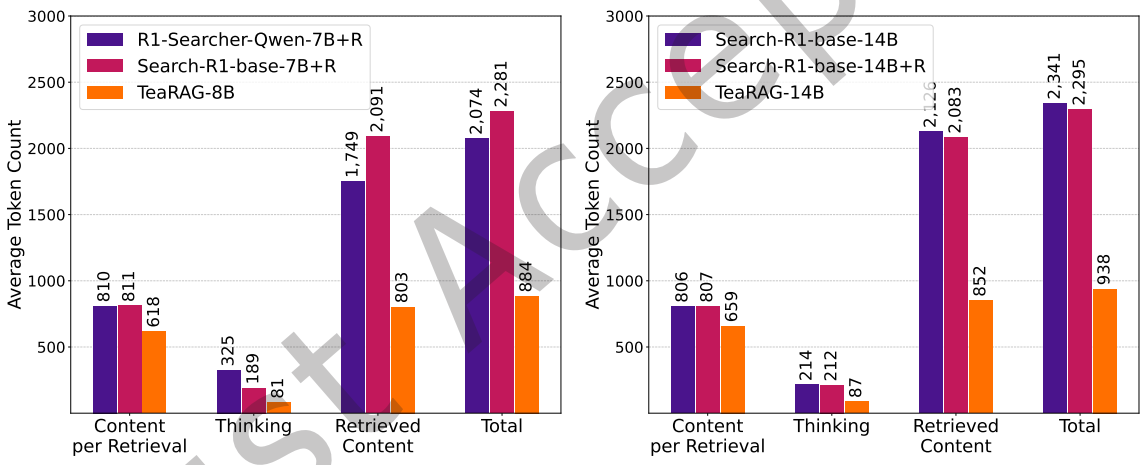


Fig. 7. Comparison of output token usage between TeaRAG and other baselines on six QA benchmarks. The figure on the left depicts the comparative results for TeaRAG-8B, while the figure on the right depicts those for TeaRAG-14B.

sensitive to model scale. Conversely, the reasoning path lengths of TeaRAG remain remarkably stable across both 8B and 14B model scales, demonstrating our method's robustness and generalizability.

5.3.2 Analysis of Token Usage of Reasoning Paths. In agentic RAG, the input prompt serves as static instructions, whereas the output contains the model's complete reasoning paths. Since input prompts are typically brief and fixed static instructions, they cannot adequately reflect the dynamic reasoning processes of LLMs. This is evidenced by the relatively modest instruction prompt token counts observed across agentic RAG methods, namely R1-Searcher-Qwen-7B (169), Search-R1-base-7B (119), TeaRAG-8B (136), Search-R1-base-14B (119), and TeaRAG-14B (134). Therefore, the analysis in this section focuses on the reasoning paths in the output tokens.

To analyze the token usage of the LLM's reasoning path more clearly, we measure four key metrics. (1) **Thinking tokens**, which are used for the LLM's thought processes like planning, problem decomposition, and summarization. (2) **Retrieved Content tokens**, which represent the token count for all externally retrieved information in the reasoning path. (3) **Total tokens**, which denote the tokens for the complete reasoning path including both Thinking tokens and Retrieved Content tokens. (4) **Content per Retrieval**, which is the average number of content tokens input to the LLM per retrieval. The results are shown in Fig. 7.

Based on these results, we can draw the following conclusions:

(1) **TeaRAG demonstrates superior token efficiency compared to the other baselines across all four metrics.** This is primarily because introducing signals from process supervision enhances the LLM's reasoning efficiency, which in turn reduces ineffective reasoning and redundant retrievals, thereby significantly improving token efficiency. Furthermore, our adoption of the KAG retrieval method also reduces the number of tokens required per retrieval.
(2) **TeaRAG's retrieval method effectively reduces the number of external content tokens per retrieval.** This is achieved by PPR to replace redundant and irrelevant chunks with high-information-density triplets. As a result, TeaRAG-8B reduces the average tokens per retrieval by 23.80% compared to Search-R1-base-7B+R.

5.4 Ablation Study

5.4.1 Effect of Context Retrieval Methods. We conduct ablation experiments on the context retrieval methods. We primarily focus on two core retrieval paradigms: iterative and hybrid search. To further understand the role of these paradigms, we design the following context retrieval variants:

- **Single-G** performs single-round graph retrieval to obtain knowledge triplets pertinent to the query, which are subsequently provided to an LLM for answer generation.
- **Single-S** conducts single-round semantic retrieval, then feeds chunks to an LLM for output.
- **Single-Con** adopts a single-round hybrid strategy combining semantic and graph retrieval without KAG construction and PPR filtering. The top-5 chunks and top-5 triplets are concatenated and passed to an LLM for answer generation.
- **Single-PPR** applies a hybrid retrieval approach incorporating semantic and graph retrieval. A KAG is constructed and refined using PPR, after which the top-5 ranked chunks and triplets are input to an LLM for answer generation.
- **TeaRAG-G** employs iterative question decomposition via an LLM, retrieving relevant knowledge triplets exclusively through graph retrieval until the final answer is produced.
- **TeaRAG-S** employs iterative question decomposition via an LLM, retrieving relevant chunks solely through semantic retrieval until the final answer is produced.
- **TeaRAG-Con** employs a hybrid retrieval approach combining semantic and graph retrieval. For each step, it directly concatenates the top-5 chunks and top-5 triplets without KAG construction and PPR filtering to form the context. An LLM then iteratively invokes this hybrid retrieval and decomposes the question until the final answer is obtained.

It is worth noting that the Single-Con and TeaRAG-Con methods use 10 information units per retrieval, while all other methods use 5. We conduct an evaluation of context retrieval methods on two base models, with the results presented in Table 8. From this, we derive the following observations:

(1) **Agentic iterative retrieval excels by decomposing complex problems.** This shows that by breaking down complex problems into simpler ones and solving them step by step, the retrieval system can better focus its information retrieval on a specific problem.
(2) **Hybrid retrieval achieves superior performance compared to using a single retrieval method.** This indicates that hybrid retrieval leverages the strengths of both semantic and graph retrieval. The fusion of

Table 8. Performance comparison of context retrieval methods on QA benchmarks. CpR represents the average number of content tokens per retrieval. The best results are highlighted in bold.

LLM	Method	Retrieval	Iterative	CpR	NQ		PopQA		HotpotQA		2Wiki		Musique		Bamboogle		Avg.	
					EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Llama3-8B	**Single-G**	**Graph**	✘	79	25.76	32.83	41.49	46.53	21.74	29.07	14.65	19.82	4.26	9.65	12.00	18.65	19.98	26.09
	Single-S	**Semantic**	✘	779	37.28	48.48	41.08	50.27	29.53	40.80	9.64	22.12	5.54	12.08	11.20	19.94	22.37	32.28
	Single-Con	**Hybrid**	✘	859	40.61	50.24	44.69	52.10	29.71	40.32	10.50	20.73	5.92	11.94	16.00	23.82	24.57	33.19
	Single-PPR	**Hybrid**	✘	650	38.89	49.52	44.72	52.84	31.42	41.69	12.59	20.58	7.65	13.72	16.80	24.06	25.35	33.73
	TeaRAG-G	**Graph**	✔	88	34.27	43.20	45.78	49.94	31.71	42.87	40.35	46.04	15.85	25.46	35.20	44.15	33.86	41.94
	TeaRAG-S	**Semantic**	✔	790	**50.36**	**59.98**	51.66	55.85	46.47	59.25	46.88	53.47	26.44	36.55	**47.20**	59.32	44.84	54.07
	TeaRAG-Con	**Hybrid**	✔	879	50.08	59.79	51.97	**56.25**	**46.66**	**59.68**	47.38	53.88	26.73	37.11	45.60	58.92	44.74	54.27
	TeaRAG	**Hybrid**	✔	618	50.06	59.71	**51.98**	56.08	46.59	59.48	**47.89**	**54.21**	**26.98**	**37.34**	**47.20**	**59.89**	**45.12**	**54.45**
Qwen2.5-14B	**Single-G**	**Graph**	✘	82	22.69	30.72	37.82	44.41	22.90	30.94	22.97	28.13	3.97	9.71	10.40	17.82	20.12	26.95
	Single-S	**Semantic**	✘	814	33.29	46.71	40.87	49.75	32.47	44.80	21.66	31.88	7.86	15.77	22.40	32.03	26.42	36.82
	Single-Con	**Hybrid**	✘	895	39.81	51.57	44.48	52.42	37.37	49.26	28.77	35.63	8.48	16.94	18.40	31.64	29.55	39.58
	Single-PPR	**Hybrid**	✘	678	40.91	52.95	46.29	53.48	37.84	49.68	29.87	36.57	9.10	17.42	22.40	34.18	31.07	40.71
	TeaRAG-G	**Graph**	✔	90	35.84	44.64	46.58	50.62	34.27	45.33	40.67	46.13	16.22	27.61	38.40	52.11	35.33	44.41
	TeaRAG-S	**Semantic**	✔	823	**51.30**	**60.98**	53.66	58.09	50.43	63.41	51.77	57.64	**28.22**	39.37	**50.40**	**65.30**	**47.63**	**57.47**
	TeaRAG-Con	**Hybrid**	✔	902	50.33	60.31	**54.13**	**58.65**	**50.71**	**63.73**	**52.36**	**58.13**	27.68	**39.41**	48.00	63.99	47.20	57.37
	TeaRAG	**Hybrid**	✔	659	50.33	60.31	53.59	58.02	49.93	63.14	52.29	57.91	27.93	39.22	**50.40**	64.07	47.41	57.11

content retrieved by the two methods further enhances the LLM's ability to answer questions, especially for non-iterative methods.

(3) **TeaRAG constructs a KAG and applies PPR-based filtering to increase the information density of retrieved content while preserving strong performance.** This is because PPR's co-occurrence-based mechanism removes irrelevant or verbose chunks and thereby further enhances accuracy. In contrast, although TeaRAG-Con utilizes hybrid retrieval, its direct concatenation of chunks and triplets increases input length and introduces extraneous noise.

(4) **Semantic retrieval outperforms graph retrieval by preserving information integrity.** This is primarily because important information within the chunks can be lost during graph construction. Additionally, the triplets' lack of sufficient context can lead to ambiguity and increase the difficulty for the LLM to comprehend.

5.4.2 Effect of Knowledge Graph Quality. To investigate the robustness of TeaRAG to knowledge graph quality, we simulate degradation by randomly pruning the triplet set $\mathcal{E}$ at varying ratios. The rationale is that extraction quality is inherently tied to LLM capability. Less capable models tend to produce sparser and less complete triplet sets, which can be approximated by progressively removing triplets from the original graph. A higher pruning ratio corresponds to a lower knowledge graph quality. The results are presented in Table 9, from which we make the following observations:

(1) **TeaRAG exhibits high robustness to knowledge graph degradation.** The PPR filtering mechanism effectively mitigates the impact of distracting noise from low-quality graphs, precluding the integration of irrelevant triplets into the LLM.

(2) **Lower knowledge graph quality correlates with an increased average token count per retrieval.** As triplet quality diminishes, the PPR mechanism filters out more graph-based results, leading to a higher retention of chunks. This confirms that TeaRAG dynamically prioritizes higher-quality retrieved content to maintain information density.

Table 9. The impact of knowledge graph quality on performance in QA benchmarks. CpR represents the average number of content tokens per retrieval. The best results are highlighted in bold.

LLM	Pruning Ratio	CpR	NQ		PopQA		HotpotQA		2Wiki		Musique		Bamboogle		Avg.	
			EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Llama3-8B	**0%**	618	50.06	59.71	51.98	56.08	46.59	59.48	**47.89**	**54.21**	**26.98**	**37.34**	47.20	59.89	45.12	**54.45**
	25%	630	49.70	59.51	51.98	56.11	**46.86**	**59.80**	47.79	53.98	26.81	37.23	48.00	59.89	**45.19**	54.42
	50%	649	49.78	59.45	**52.02**	**56.18**	46.37	59.37	47.42	53.69	26.60	37.10	**48.80**	**60.36**	45.16	54.36
	75%	681	50.06	59.52	51.73	55.86	46.51	59.47	47.59	53.90	26.60	37.02	46.40	59.14	44.81	54.15
	100%	790	**50.36**	**59.98**	51.66	55.85	46.47	59.25	46.88	53.47	26.44	36.55	47.20	59.32	44.84	54.07
Qwen2.5-14B	**0%**	659	50.33	60.31	53.59	58.02	49.93	63.14	**52.29**	57.91	27.93	39.22	50.40	64.07	47.41	57.11
	25%	660	50.47	60.50	53.45	57.88	50.06	63.24	52.19	**57.96**	27.55	39.09	50.40	64.26	47.35	57.16
	50%	678	51.05	60.97	53.49	57.94	50.11	63.06	52.08	57.77	27.31	39.12	51.20	65.70	47.54	57.43
	75%	710	51.00	60.76	53.26	57.71	50.40	63.29	51.94	57.81	27.97	**39.57**	**52.00**	**65.80**	**47.76**	**57.49**
	100%	823	**51.30**	**60.98**	**53.66**	**58.09**	**50.43**	**63.41**	51.77	57.64	**28.22**	39.37	50.40	65.30	47.63	57.47

Table 10. Performance comparison of models trained with different rewards. # Pairs indicates the number of preference pairs used for training. # Steps indicates the number of reasoning path steps. The best results are highlighted in bold.

Iteration	Outcome	Format	Process	# Pairs	# Steps	NQ		PopQA		HotpotQA		2Wiki		Musique		Bamboogle		Avg.	
						EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
SFT	✘	✘	✘	-	1.47	38.56	47.46	46.56	50.67	28.86	38.23	26.39	31.99	18.99	27.57	36.00	47.96	32.56	40.65
1	✔	✘	✘	8,886	1.48	48.39	57.57	50.92	55.37	43.97	55.82	45.52	51.93	24.33	33.69	**44.00**	56.91	42.85	51.88
	✔	✔	✘	9,198	1.48	48.42	57.49	51.22	55.43	43.58	55.79	**46.09**	**51.97**	23.91	33.30	**44.00**	**58.23**	42.87	52.03
	✔	✔	✔	8,760	1.38	**49.22**	**58.13**	**51.97**	**56.27**	**44.02**	**56.25**	45.28	51.34	**25.69**	**35.22**	41.60	56.18	**42.96**	**52.23**
2	✔	✘	✘	5,024	1.57	42.11	49.42	46.30	49.76	39.47	49.18	37.76	40.86	17.91	22.62	39.20	46.37	37.12	43.03
	✔	✔	✘	4,576	1.76	48.64	57.94	50.75	55.02	45.25	57.19	44.78	49.99	23.13	32.51	**47.20**	**59.07**	43.29	51.95
	✔	✔	✔	4,760	1.35	**48.89**	**58.98**	**51.28**	**55.84**	**45.62**	**58.88**	**46.72**	**53.48**	**26.02**	**36.69**	43.20	54.58	**43.62**	**53.07**
3	✔	✔	✘	3,832	2.13	48.42	57.61	50.19	54.30	39.91	49.61	39.42	43.14	18.82	25.28	41.60	50.45	39.73	46.73
	✔	✔	✔	2,941	1.31	**50.06**	**59.71**	**51.98**	**56.08**	**46.59**	**59.48**	**47.89**	**54.21**	**26.98**	**37.34**	**47.20**	**59.89**	**45.12**	**54.45**

5.4.3 Effect of Process Rewards. To analyze the advantages of process rewards, we perform an ablation study specifically on process rewards. We validate our approach on Llama3-8B-Instruct, considering three distinct schemes: using only **Outcome** reward, using **Outcome-Format** reward, and using **Outcome-Format-Process** reward. We train models with each reward type for multiple rounds. The results are shown in Table 10. We observe the following phenomena:

(1) **Process rewards effectively enhance model performance.** By providing rewards for each intermediate step of the reasoning path, the LLM can learn from high-quality reasoning examples.
(2) **Process rewards reduce reasoning steps and increase reasoning efficiency.** The reward mechanism penalizes unnecessary reasoning steps that are not instrumental to problem-solving. In contrast, models trained without process rewards exhibit a tendency to gradually increase their reasoning step count. This phenomenon is attributed to the lack of intermediate supervision, which can reinforce redundant reasoning paths that ultimately lead to the correct result.
(3) **Process rewards make training more stable.** With only outcome rewards, performance collapses by the second training round. The model prefers correct answers via incorrectly formatted paths, undermining its

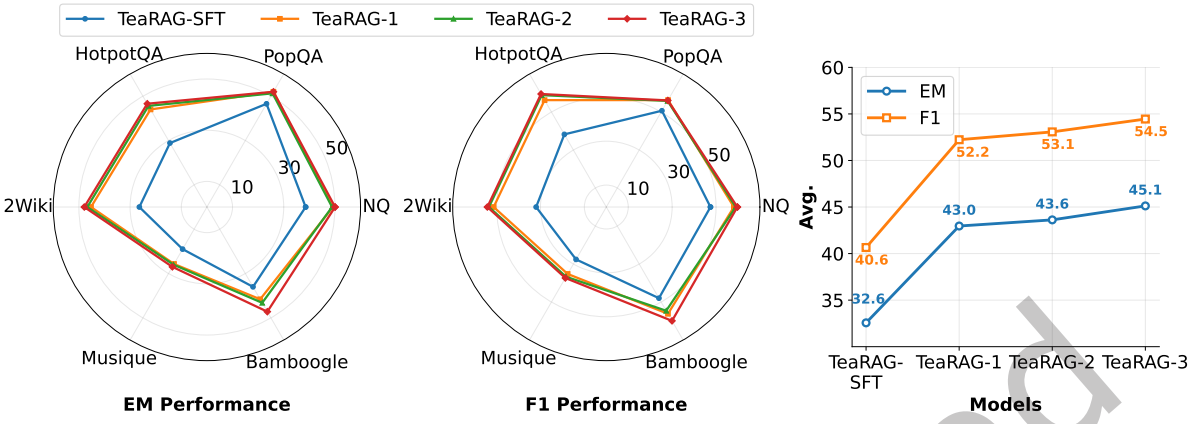


Fig. 8. Performance of TeaRAG with Llama3-8B-Instruct across IP-DPO iterations. The left and middle figures show the EM and F1 scores of TeaRAG on six datasets, respectively. The right figure displays the average EM and F1 scores of TeaRAG across the six datasets.

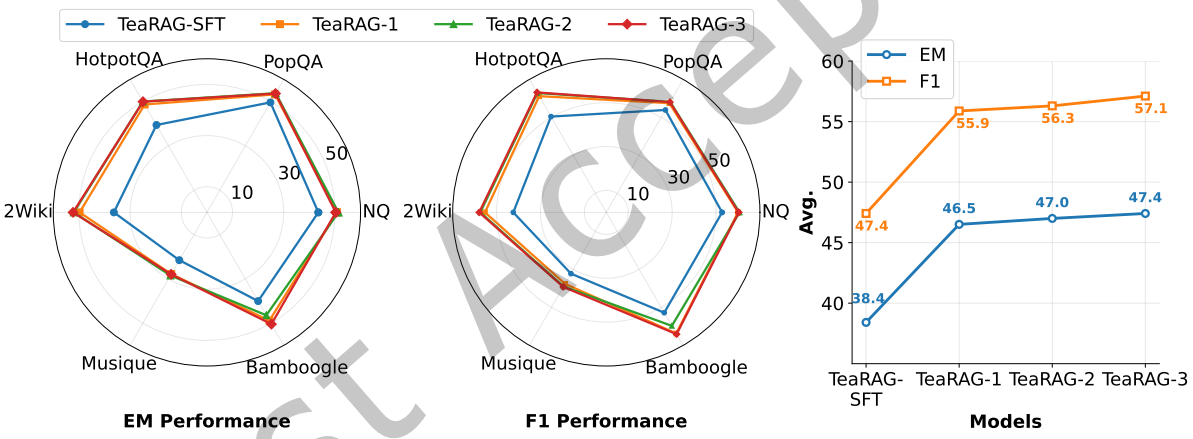


Fig. 9. Performance of TeaRAG with Qwen2.5-14B-Instruct across IP-DPO iterations. The left and middle figures show the EM and F1 scores of TeaRAG on six datasets, respectively. The right figure displays the average EM and F1 scores of TeaRAG across the six datasets.

ability to follow the intended reasoning structure. Similarly, when using both outcome and format rewards, collapse occurs by the third round. The model still rewards correct answers produced through flawed logic. This failure mode is especially common in binary-choice comparison tasks, leading it to internalize misguided preferences for certain reasoning paths. Process rewards prevent this by constructing preference pairs under stricter criteria, blocking these spurious preferences and stabilizing training.

5.4.4 Effect of IP-DPO Iterations. We investigate the impact of IP-DPO iterations by evaluating models across all training stages. TeaRAG-SFT denotes the model trained only with SFT. TeaRAG-1, TeaRAG-2, and TeaRAG-3

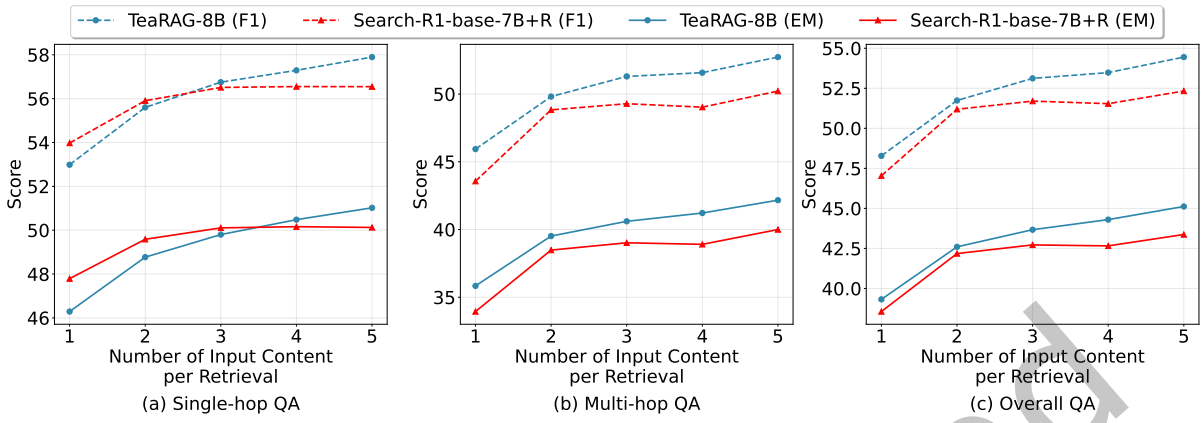


Fig. 10. Comparative performance of TeaRAG-8B and Search-R1-base-7B+R across varying numbers of input content per retrieval.

refer to the models after 1, 2, and 3 rounds of DPO training, respectively. The results are presented in Figure 8 and Figure 9. We can draw the following conclusions:

(1) **The performance of TeaRAG exhibits continuous enhancement as the number of IP-DPO rounds increases.** The underlying mechanism is that each round of optimization leverages the outputs from the model of the preceding round. This enables the model to sequentially master and refine its reasoning paradigms.
(2) **The performance gains from IP-DPO exhibit diminishing returns with additional rounds.** This occurs because the model's reasoning capabilities begin to converge, establishing a stable inference pattern. After many rounds of iteration, the model consistently produces correct responses for many questions, causing the preference data to provide a diminishing learning signal and thus limiting further improvement.
(3) **IP-DPO demonstrates consistent and stable improvements across models of varying scales and families.** This consistent success demonstrates IP-DPO's broad applicability for enhancing reasoning across diverse LLMs.

5.5 Hyper-Parameter Analysis

5.5.1 Number of Input Contents per Retrieval. In this section, we investigate the impact of the number of input contents per retrieval on method performance. We compare the EM and F1 scores of TeaRAG-8B and Search-R1-base-7B+R, as well as TeaRAG-14B and Search-R1-base-14B+R. The experimental results are shown in Fig. 10 and Fig. 11. We have the following observations:

(1) **More retrieved content improves performance, with TeaRAG consistently outperforming the baseline.** This indicates that TeaRAG is robust to variations in the number of input contents per retrieval and can maintain strong performance across different settings.
(2) **When the number of input contents per retrieval is small, TeaRAG cannot fully leverage the capabilities of LLMs. When the number is larger (≥ 3), it can achieve outstanding results.** This is mainly because TeaRAG 's training leverages PPR based on the co-occurrence mechanism. LLMs tend to focus on co-occurring data features, and when the number of input contents per retrieval is small, co-occurrence is difficult to achieve. This leads to a mismatch between inference and training, resulting in input distribution drift. When the number is sufficiently large, the information filtered by PPR has a higher information density, and at the same time, the LLM can exploit the co-occurrence in the data to achieve better performance.

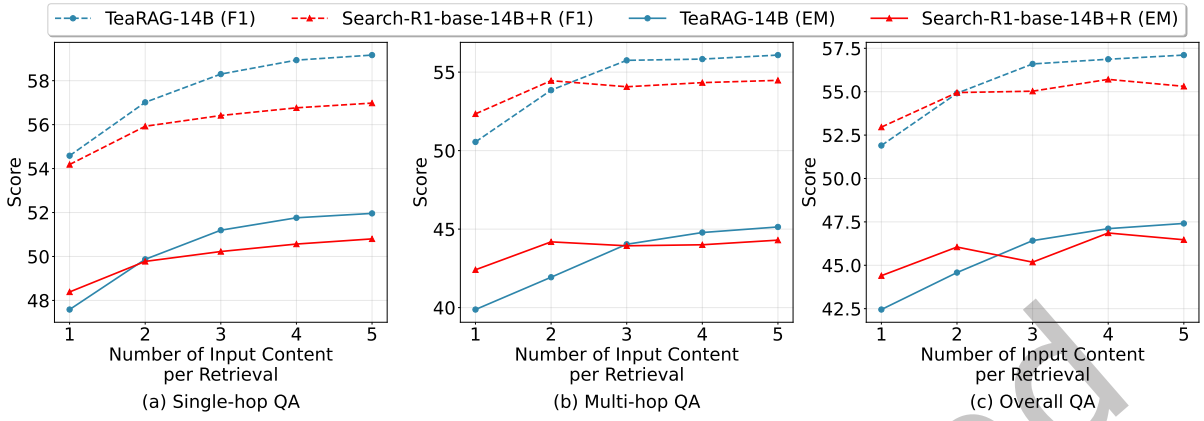


Fig. 11. Comparative performance of TeaRAG-14B and Search-R1-base-14B+R across varying numbers of input content per retrieval.

(3) **TeaRAG is more scalable, with a more significant performance boost as the number of input contents per retrieval increases.** This is due to the construction of KAG and effective PPR filtering, which introduce triplets to increase information density and accuracy, thus reducing irrelevant content for each input. In contrast, the baseline method typically hits a performance bottleneck once the number of inputs reaches 3, as traditional retrieval-and-rerank approaches still cannot avoid interference from irrelevant information in chunks.

5.5.2 Effects of Hyperparameter α in PPR. In this section, we systematically investigate the influence of the PPR hyperparameter α, which controls the focus of retrieved content. When α is small, PPR emphasizes the personalization vector, i.e., semantic information relevant to the query. When α is large, it prioritizes the structured information of co-occurrence links. The experimental results are shown in Fig. 12. We have the following observations:

(1) **TeaRAG is robust to changes in α.** The performance of TeaRAG varies little for α between 0.1 and 0.7. This robustness stems from KAG, which is constructed from the outputs of two retrieval methods, ensuring that its content is already highly relevant to the query.
(2) **TeaRAG achieves better performance by balancing query relevance and co-occurrence relations.** TeaRAG typically performs best when α is in the range 0.3 to 0.7. This indicates that PPR filtering effectively combines both factors to enhance information density and accuracy.
(3) **Greater emphasis on co-occurrence structure reduces the average number of content tokens per retrieval.** This is because, as co-occurrence structure receives more weight, information from triplet nodes connected to entities and chunks is more activated and ranked higher, thereby reducing the token count. This reveals a trade-off where despite fewer input tokens, missing important background leads to a significant performance drop, e.g., at $\alpha = 0.9$.

5.5.3 Effects of Generation Temperature. Because the sampling temperature has a pronounced impact on LLM performance [4], we vary the generation temperature for TeaRAG-8B and Search-R1-base-7B+R to probe robustness. The results are shown in Fig. 13. We have the following observations:

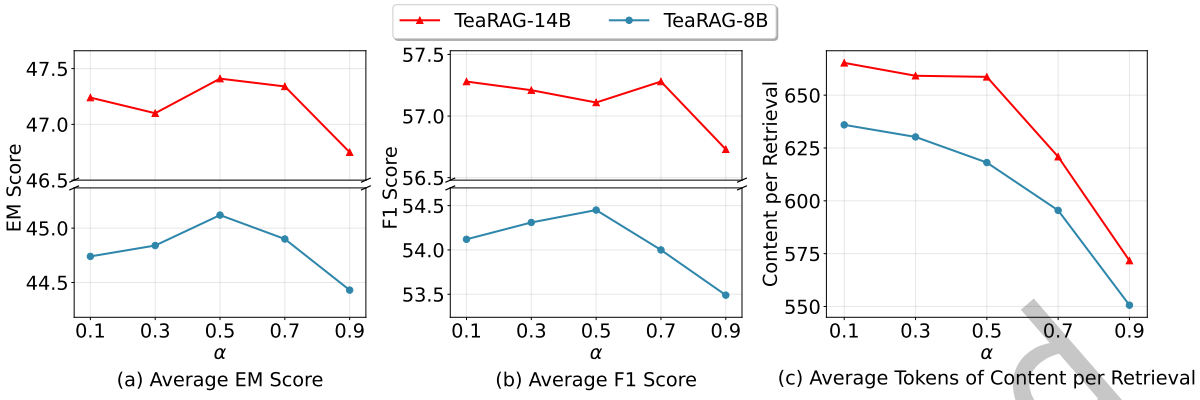


Fig. 12. Performance and the average number of content tokens per retrieval on six QA benchmarks for TeaRAG-8B and TeaRAG-14B across varying α in PPR.

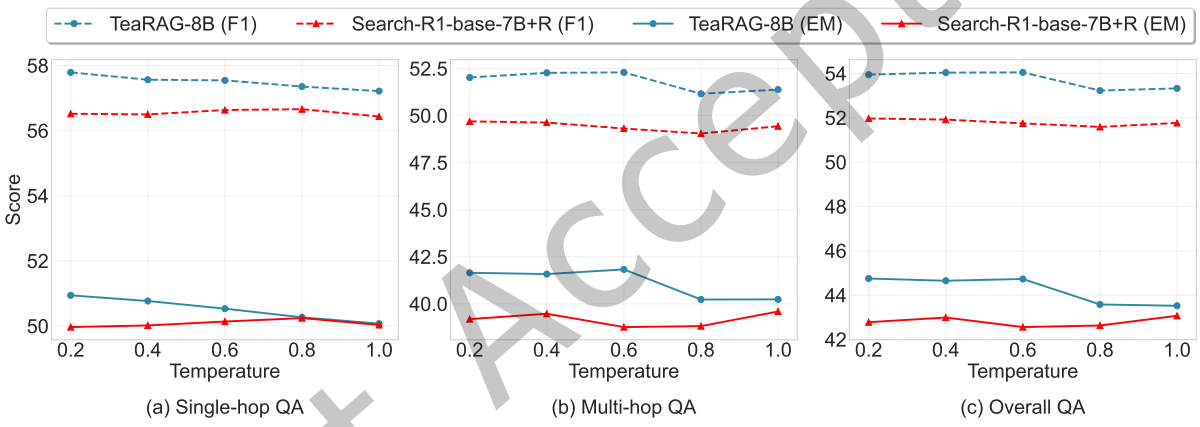


Fig. 13. Performance of TeaRAG-8B with Search-R1-base-7B+R across generation temperatures.

(1) **Agentic RAG methods are not sensitive to temperature.** This is because agentic RAG not only requires the LLM to generate content, but also to ground its generation in retrieved external information, which reduces uncertainty.
(2) **TeaRAG-8B consistently outperforms Search-R1-base-7B+R across different temperatures, further demonstrating the effectiveness and robustness of TeaRAG.** We performed a two-sided paired t-test on the overall QA results, and $p < 0.05$ indicates that our method is significantly better than Search-R1-base-7B+R.
(3) **As temperature increases, the performance of TeaRAG-8B exhibits a slight decline.** A likely reason is that higher temperature induces more diverse reasoning paths, which increases variance and can occasionally lead to deviations from the retrieved evidence or extra, unnecessary steps, thereby reducing answer accuracy and consistency.

Table 11. Training overhead across methods on 8 NVIDIA A100 (80G) GPUs.

Method	Overall Training Time	Training Time	Inference Time	Reward Time	Training Steps	Time/Step	Memory Usage per GPU
TeaRAG-8B	681m	85m	376m	220m	1,688	0.40m	42G
Search-R1-base-7B	2,944m	1,320m	1,624m	-	1,005	2.93m	79G
TeaRAG-14B	752m	115m	417m	220m	1,513	0.49m	61G
Search-R1-base-14B	4,958m	2,462m	2,496m	-	1,005	4.93m	80G

Table 12. Inference efficiency on the 2WikiMultiHopQA dev dataset (12, 576 questions).

Method	Semantic Retrieval	Graph Retrieval	KAG+PPR	Retrieval Time	Generation Time	Overall Time
TeaRAG-8B (Sequential)	270s	282s	24s	578s	482s	1,061s
TeaRAG-8B (Parallel)	305s	301s	22s	327s	490s	820s
Search-R1-base-7B+R	601s	-	-	601s	1,641s	2,243s
TeaRAG-14B (Sequential)	281s	334s	22s	639s	495s	1,136s
TeaRAG-14B (Parallel)	314s	345s	22s	368s	497s	868s
Search-R1-base-14B+R	525s	-	-	525s	2,655s	3,181s

5.6 Training and Inference Efficiency Analysis

In this section, we provide an in-depth analysis of the training and inference efficiency of TeaRAG. For the training overhead, the statistics are summarized in the Table 11. First, TeaRAG requires less total training time compared to Search-R1, primarily because the IP-DPO framework decouples the training and sampling stages, thereby enhancing overall training efficiency. Furthermore, TeaRAG achieves lower memory consumption through LoRA training and without reliance on auxiliary models such as PPO critics.

Furthermore, we evaluate TeaRAG's inference efficiency against Search-R1 on the 2WikiMultiHopQA dataset. We implement two retrieval strategies for dual-path retrieval in TeaRAG-8B and TeaRAG-14B, namely sequential retrieval and parallel retrieval. The inference time for each stage is presented in Table 12. Our analysis yields three key observations:

(1) **The overall inference time of TeaRAG is significantly lower than that of Search-R1.** This reduction can be attributed to the IP-DPO training method, which reduces the reasoning steps, thereby avoiding overthinking and redundant retrieval steps. Furthermore, the incorporation of knowledge triplets enhances information density and reduces token length. Consequently, the generation time is substantially reduced.
(2) **Parallel retrieval mitigates the latency overhead introduced by graph retrieval.** Since the semantic retrieval and graph retrieval processes are decoupled, they can be executed in parallel. Consequently, the overall retrieval latency is bounded by the maximum of the two individual branch latencies, rather than their sum. Parallel execution thereby introduces only marginal additional system-parallel overhead compared to a single-path retrieval baseline.
(3) **Both KAG construction and PPR filtering are efficient.** The two operations collectively account for a negligible fraction of the total inference latency, requiring on average only 1 ms per reasoning step. This confirms that TeaRAG can efficiently leverage co-occurrence relationships to produce higher–information-density retrieval content.

Finally, we measure the GPU and main memory usage of the two retrieval methods. The semantic retriever requires 2.6 GB of GPU memory and 5.8 GB of main memory, whereas the graph retriever requires 2.6 GB of GPU memory and 13.1 GB of main memory. These results suggest that dual-path retrieval incurs only moderate resource overhead, which is acceptable given the reduction in the number of generated tokens and the resulting improvement in generation efficiency.

Table 13. Case comparison between TeaRAG-8B and Search-R1-base-7B+R.

Question: Where was the place of death of Alexander Carl Otto Westphal's father?
Ground Truth: Kreuzlingen
TeaRAG-8B: *Step 1:* *Important entity:* Alexander Carl Otto Westphal *Subquery:* Who was the father of Alexander Carl Otto Westphal? <Reference> Evidence 1: Title: "Alexander Carl Otto Westphal" Text: number of significant scientists, including Otto Lowenstein (1889-1965), with whom he was co-author of "Experimentelle und klinische Studien zur Physiologie und Pathologie der Pupillenbewegungen, mit besonderer Berücksichtigung der Schizophrenie" ... Evidence 2: Title: "Alexander Carl Otto Westphal" Text: Alexander Carl Otto Westphal Alexander Carl Otto Westphal (18 May 1863, Berlin – 9 January 1941, Bonn) was a German neurologist and psychiatrist... Evidence 3: Triplet: Alexander Carl Otto Westphal father Carl Friedrich Otto Westphal Evidence 4: Triplet: Alexander Carl Otto Westphal son of Carl Friedrich Otto Westphal Evidence 5: Title: "Carl Friedrich Otto Westphal" Text: ... Carl Friedrich Otto Westphal Karl Friedrich Otto Westphal (23 March 1833, in Berlin – 27 January 1890, in Kreuzlingen) was a German psychiatrist from Berlin. He was the son of Otto Carl Friedrich Westphal (1800–1879) and Karoline Friederike Heine and the father of Alexander Karl Otto Westphal (1863-1941)... </Reference> *Summary:* The fact Alexander Carl Otto Westphal's father Carl Friedrich Otto Westphal died in Kreuzlingen logically implies the answer: Kreuzlingen. *Final answer:* Kreuzlingen
Search-R1-base-7B+R: <think> I need to find the place of death of Alexander Carl Otto Westphal's father. I'll search for it. </think> <search> Alexander Carl Otto Westphal's father </search> <information> Doc 1 (Title: "Carl Friedrich Otto Westphal") Carl Friedrich Otto Westphal Karl Friedrich Otto Westphal (23 March 1833, in Berlin – 27 January 1890, in Kreuzlingen) was a German psychiatrist from Berlin... Doc 2 (Title: "Alexander Carl Otto Westphal") Alexander Carl Otto Westphal Alexander Carl Otto Westphal (18 May 1863, Berlin – 9 January 1941, Bonn) was a German neurologist and psychiatrist... [3 more docs...] </information> <think> I found out that Alexander Carl Otto Westphal's father is Carl Friedrich Otto Westphal. Now I need to find the place of death of his father, Otto Carl Friedrich Westphal. </think> <search> place of death of Otto Carl Friedrich Westphal </search> <information> Doc 1 (Title: "Carl Friedrich Otto Westphal") Carl Friedrich Otto Westphal Karl Friedrich Otto Westphal (23 March 1833, in Berlin – 27 January 1890, in Kreuzlingen) was a German psychiatrist from Berlin. He was the son of Otto Carl Friedrich Westphal... [4 more docs...] </information> <think> I found out that Otto Carl Friedrich Westphal died in Berlin. Now I need to find the place of death of his father, Otto Carl Friedrich Westphal. </think> <search> place of death of Otto Carl Friedrich Westphal </search> <information> [5 docs with similar information...]</information> <think> I found out that Otto Carl Friedrich Westphal died in Berlin. Now I can provide the answer. </think> <answer> Berlin </answer>

5.7 Case Study

In this section, we present a case study of TeaRAG-8B and Search-R1-base-7B+R to gain a clearer understanding of TeaRAG's advantages. As shown in Table 13, we observe two advantages of TeaRAG-8B. First, the content retrieved by TeaRAG-8B effectively increases information density by introducing knowledge triplets, and it further strengthens confidence in the correct information by leveraging the co-occurrence between chunks and triplets. In addition, TeaRAG-8B effectively reduces the number of iterations in the reasoning path, fully leveraging available information to arrive at the correct answer. In contrast, Search-R1-base-7B+R pulls in a large amount of document content, which distracts the model from effectively capturing key information to obtain

the correct answer, and the model exhibits overthinking and redundant retrieval, leading to poor overall token efficiency.

6 Conclusion and Future Work

In this work, we explore a token-efficient agentic RAG framework that aims to improve the token utilization of reasoning paths while performing agentic RAG tasks. We observe that better token efficiency relies on increasing the information density of each retrieved content and reducing the number of reasoning iterations. To this end, we propose TeaRAG. TeaRAG provides a fully automated agentic pipeline, including important entity recognition, subquery generation, context retrieval, summary generation, and final answer generation. To raise the information density per retrieval step, we incorporate high-information-density knowledge triplets and adopt a hybrid retrieval strategy that combines semantic and graph retrieval. We further construct a KAG and apply co-occurrence-based PPR to preserve rich context and boost information density. To reduce the number of reasoning iterations, we introduce IP-DPO, which uses process-aware rewards to supervise the reasoning path, preventing overthinking and redundant retrieval. Extensive experiments demonstrate the effectiveness of our approach and its superior token efficiency.

To further broaden the framework's adaptability to domains lacking detailed evidence annotations, our future work will focus on extending process supervision beyond the reliance on human-annotated gold evidence. We plan to explore two directions to verify reasoning paths: (1) leveraging advanced LLMs as automated evaluators (LLM-as-a-Judge) to assess intermediate steps, and (2) training a specialized process reward model to provide step-by-step supervision.

Acknowledgments

This work was supported in part by the grants from National Science and Technology Major Project (No. 2023ZD0121104), and the Anhui Natural Science Foundation (No. 2508085ZD006). Besides, this research was partially supported by National Natural Science Foundation of China (No.62502404), Hong Kong Research Grants Council (Research Impact Fund No.R1015-23, Collaborative Research Fund No.C1043-24GF, General Research Fund No. 11218325), Institute of Digital Medicine of City University of Hong Kong (No.9229503), Huawei (Huawei Innovation Research Program), Tencent (Tencent Rhino-Bird Focused Research Program, Tencent University Cooperation Project), Kuaishou (CCF-Kuaishou Large Model Explorer Fund No. 2025008, Kuaishou University Cooperation Project), Didi (CCF-Didi Gaia Scholars Research Fund), and Bytedance.

References

[1] Nicholas Alonso and Beren Millidge. 2024. Mixture-of-PageRanks: Replacing Long-Context with Real-Time, Sparse GraphRAG. *arXiv preprint arXiv:2412.06078* (2024).

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *NeurIPS* 33 (2020), 1877–1901.

[4] Nikhil Chandak, Shashwat Goel, and Ameya Prabhu. 2025. Incorrect Baseline Evaluations Call into Question Recent LLM-RL Claims. https://safe-lip-9a8.notion.site/Incorrect-Baseline-Evaluations-Call-into-Question-Recent-LLM-RL-Claims-2012f1fbf0ee8094ab8ded1953c15a37?pvs=4. Notion Blog.

[5] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the association for computational linguistics: ACL 2024*. 2318–2335.

[6] Liyi Chen, Panrong Tong, Zhongming Jin, Ying Sun, Jieping Ye, and Hui Xiong. [n. d.]. Plan-on-Graph: Self-Correcting Adaptive Planning of Large Language Model on Knowledge Graphs. In *NeurIPS*.

[7] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. 2024. Do not think that much for 2+ 3=? on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187* (2024).

[8] Yuhao Chen, Shuochen Liu, Yuanjie Lyu, Chao Zhang, Jiayao Shi, and Tong Xu. 2025. Xiangqi-r1: Enhancing spatial strategic reasoning in llms for chinese chess via reinforcement learning. *arXiv preprint arXiv:2507.12215* (2025).
[9] Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, et al. 2025. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks. *arXiv preprint arXiv:2502.08235* (2025).
[10] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
[11] Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. 2025. KiRAG: Knowledge-Driven Iterative Retriever for Enhancing Retrieval-Augmented Generation. *arXiv preprint arXiv:2502.18397* (2025).
[12] Jiaxuan Gao, Wei Fu, Minyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. 2025. Beyond Ten Turns: Unlocking Long-Horizon Agentic Search with Large-Scale Asynchronous RL. *arXiv preprint arXiv:2508.07976* (2025).
[13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2 (2023).
[14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
[15] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint arXiv:2410.05779* (2024).
[16] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. In *NeurIPS*.
[17] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From rag to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802* (2025).
[18] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *NeurIPS* 37 (2024), 132876–132907.
[19] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *COLING*. 6609–6625.
[20] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
[21] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. 2025. Reinforce++: An efficient rlhf algorithm with robustness to both prompt and reward models. *arXiv preprint arXiv:2501.03262* (2025).
[22] Hamish Ivison, Yizhong Wang, Jiacheng Liu, Zeqiu Wu, Valentina Pyatkin, Nathan Lambert, Noah A Smith, Yejin Choi, and Hanna Hajishirzi. 2024. Unpacking dpo and ppo: Disentangling best practices for learning from preference feedback. *NeurIPS* 37 (2024), 36602–36633.
[23] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720* (2024).
[24] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. [n. d.]. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *EMNLP*.
[25] Pengcheng Jiang, Lang Cao, Ruike Zhu, Minhao Jiang, Yunyi Zhang, Jimeng Sun, and Jiawei Han. 2025. RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation. *arXiv preprint arXiv:2502.10996* (2025).
[26] Pengcheng Jiang, Xueqiang Xu, Jiacheng Lin, Jinfeng Xiao, Zifeng Wang, Jimeng Sun, and Jiawei Han. 2025. s3: You Don't Need That Much Data to Train a Search Agent via RL. *arXiv preprint arXiv:2505.14146* (2025).
[27] Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. 2025. An Empirical Study on Reinforcement Learning for Reasoning-Search Interleaved LLM Agents. *arXiv preprint arXiv:2505.15117* (2025).
[28] Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
[29] Jiajie Jin, Yutao Zhu, Zhicheng Dou, Guanting Dong, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, and Ji-Rong Wen. 2025. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In *Companion Proceedings of the ACM on Web Conference 2025*. 737–740.
[30] Adam Tauman Kalai, Ofir Nachum, Santosh S Vempala, and Edwin Zhang. 2025. Why language models hallucinate. *arXiv preprint arXiv:2509.04664* (2025).
[31] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering.. In *EMNLP*. 6769–6781.
[32] Saeed Khaki, JinJin Li, Lan Ma, Liu Yang, and Prathap Ramachandra. 2024. RS-DPO: A Hybrid Rejection Sampling and Direct Preference Optimization Method for Alignment of Large Language Models. In *Findings of NAACL*. 1665–1680.

[33] Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. [n. d.]. Understanding the Effects of RLHF on LLM Generalisation and Diversity. In *The Twelfth International Conference on Learning Representations*.
[34] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *TACL* 7 (2019), 453–466.
[35] Hanyu Lai, Xiao Liu, Hao Yu, Yifan Xu, Iat Long Iong, Shuntian Yao, Aohan Zeng, Zhengxiao Du, Yuxiao Dong, and Jie Tang. 2025. WebGLM: Towards an Efficient and Reliable Web-Enhanced Question-Answering System. *ACM Trans. Inf. Syst.* 43, 5, Article 122 (July 2025), 43 pages. doi:10.1145/3729421
[36] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS* 33 (2020), 9459–9474.
[37] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366* (2025).
[38] Xiaopeng Li, Lixin Su, Pengyue Jia, Suqi Cheng, Junfeng Wang, Dawei Yin, and Xiangyu Zhao. 2025. Agent4Ranking: Semantic Robust Ranking via Personalized Query Rewriting Using Multi-agent LLMs. *TOIS* (July 2025). doi:10.1145/3749099 Just Accepted.
[39] Lei Liang, Zhongpu Bo, Zhengke Gui, Zhongshu Zhu, Ling Zhong, Peilong Zhao, Mengshu Sun, Zhiqiang Zhang, Jun Zhou, Wenguang Chen, et al. 2025. Kag: Boosting llms in professional domains via knowledge augmented generation. In *Companion Proceedings of the ACM on Web Conference 2025*. 334–343.
[40] Shuochen Liu, Pengfei Luo, Chao Zhang, Yuhao Chen, Haotian Zhang, Qi Liu, Xin Kou, Tong Xu, and Enhong Chen. 2026. Look as You Think: Unifying Reasoning and Visual Evidence Attribution for Verifiable Document RAG via Reinforcement Learning. In *AAAI*, Vol. 40. 32159–32167.
[41] Shuochen Liu, Junyi Zhu, Long Shu, Junda Lin, Yuhao Chen, Haotian Zhang, Chao Zhang, Derong Xu, Jia Li, Bo Tang, et al. 2026. Perma: Benchmarking personalized memory agents via event-driven preference and realistic task environments. *arXiv preprint arXiv:2603.23231* (2026).
[42] Haoran Luo, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, Meina Song, Xiaobao Wu, Yifan Zhu, Luu Anh Tuan, et al. 2025. Graph-R1: Towards Agentic GraphRAG Framework via End-to-end Reinforcement Learning. *arXiv preprint arXiv:2507.21892* (2025).
[43] Yuanjie Lyu, Zhiyu Li, Simin Niu, Feiyu Xiong, Bo Tang, Wenjin Wang, Hao Wu, Huanyong Liu, Tong Xu, and Enhong Chen. 2025. Crud-rag: A comprehensive chinese benchmark for retrieval-augmented generation of large language models. *TOIS* 43, 2 (2025), 1–32.
[44] Yuanjie Lyu, Zihan Niu, Zheyong Xie, Chao Zhang, Tong Xu, Yang Wang, and Enhong Chen. 2024. Retrieve-Plan-Generation: An Iterative Planning and Answering Framework for Knowledge-Intensive LLM Generation. In *EMNLP*. 4683–4702.
[45] Yuanjie Lyu, Chengyu Wang, Jun Huang, and Tong Xu. 2025. From correction to mastery: Reinforced distillation of large language model agents. *arXiv preprint arXiv:2509.14257* (2025).
[46] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. *arXiv preprint arXiv:2407.10805* (2024).
[47] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *EMNLP*. 5303–5315.
[48] Iain Mackie, Ivan Sekulic, Shubham Chatterjee, Jeffrey Dalton, and Fabio Crestani. 2023. GRM: generative relevance modeling using relevance-aware sample estimation for document retrieval. *arXiv preprint arXiv:2306.09938* (2023).
[49] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *ACL*. 9802–9822.
[50] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *NeurIPS* 35 (2022), 27730–27744.
[51] Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddartha Naidu, and Colin White. 2024. Smaug: Fixing failure modes of preference optimisation with dpo-positive. *arXiv preprint arXiv:2402.13228* (2024).
[52] Richard Yuanzhe Pang, Weizhe Yuan, He He, Kyunghyun Cho, Sainbayar Sukhbaatar, and Jason Weston. 2024. Iterative reasoning preference optimization. *NeurIPS* 37 (2024), 116617–116637.
[53] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. [n. d.]. Measuring and Narrowing the Compositionality Gap in Language Models. In *EMNLP*.
[54] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS* 36 (2023), 53728–53741.
[55] Bhaskarjit Sarmah, Dhagash Mehta, Benika Hall, Rohan Rao, Sunil Patel, and Stefano Pasquali. 2024. Hybridrag: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. In *Proceedings of the 5th ACM International Conference on AI in Finance*. 608–616.

[56] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
[57] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
[58] Yaorui Shi, Sihang Li, Chang Wu, Zhiyuan Liu, Junfeng Fang, Hengxing Cai, An Zhang, and Xiang Wang. 2025. Search and Refine During Think: Autonomous Retrieval-Augmented Reasoning of LLMs. *arXiv preprint arXiv:2505.11277* (2025).
[59] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2503.05592* (2025).
[60] Huatong Song, Jinhao Jiang, Wenqing Tian, Zhipeng Chen, Yuhuan Wu, Jiahao Zhao, Yingqian Min, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-Searcher++: Incentivizing the Dynamic Knowledge Acquisition of LLMs via Reinforcement Learning. *arXiv preprint arXiv:2505.17005* (2025).
[61] Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Fei Huang, and Yan Zhang. 2025. Zerosearch: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588* (2025).
[62] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. [n. d.]. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*.
[63] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. ♫ MuSiQue: Multihop Questions via Single-hop Question Composition. *TACL* 10 (2022), 539–554.
[64] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *ACL*.
[65] Songjun Tu, Jiahao Lin, Xiangyu Tian, Qichao Zhang, Linjing Li, Yuqian Fu, Nan Xu, Wei He, Xiangyuan Lan, Dongmei Jiang, et al. 2025. Enhancing LLM Reasoning with Iterative DPO: A Comprehensive Empirical Investigation. *arXiv preprint arXiv:2503.12854* (2025).
[66] Prakhar Verma, Sukruta Prakash Midigeshi, Gaurav Sinha, Arno Solin, Nagarajan Natarajan, and Amit Sharma. 2024. Plan* rag: Efficient test-time planning for retrieval augmented generation. *arXiv preprint arXiv:2410.20753* (2024).
[67] Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. 2025. Acting Less is Reasoning More! Teaching Model to Act Efficiently. *arXiv preprint arXiv:2504.14870* (2025).
[68] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *FCS* 18, 6 (2024), 186345.
[69] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).
[70] Shuting Wang, Xin Yu, Mang Wang, Weipeng Chen, Yutao Zhu, and Zhicheng Dou. 2025. RichRAG: Crafting Rich Responses for Multi-faceted Queries in Retrieval-Augmented Generation. In *COLING*. 11317–11333.
[71] Ziliang Wang, Xuhui Zheng, Kang An, Cijun Ouyang, Jialu Cai, Yuhang Wang, and Yichao Wu. 2025. StepSearch: Igniting LLMs Search Ability via Step-Wise Proximal Policy Optimization. *arXiv preprint arXiv:2505.15107* (2025).
[72] Jinbo Wen, Cheng Su, Jiawen Kang, Jiangtian Nie, Yang Zhang, Jianhang Tang, Dusit Niyato, and Chau Yuen. 2025. HybridRAG-based LLM Agents for Low-Carbon Optimization in Low-Altitude Economy Networks. *arXiv preprint arXiv:2506.15947* (2025).
[73] Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang, Tong Zhang, Caiming Xiong, et al. 2025. A minimalist approach to llm reasoning: from rejection sampling to reinforce. *arXiv preprint arXiv:2504.11343* (2025).
[74] Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang, and Enhong Chen. 2024. Large language models for generative information extraction: A survey. *FCS* 18, 6 (2024), 186357.
[75] Derong Xu, Pengyue Jia, Xiaopeng Li, Yingyi Zhang, Maolin Wang, Qidong Liu, Xiangyu Zhao, Yichao Wang, Huifeng Guo, Ruiming Tang, et al. 2025. Align-GRAG: Reasoning-Guided Dual Alignment for Graph Retrieval-Augmented Generation. *arXiv preprint arXiv:2505.16237* (2025).
[76] Derong Xu, Tong Xu, Shiwei Wu, Jingbo Zhou, and Enhong Chen. 2022. Relation-enhanced negative sampling for multimodal knowledge graph completion. In *MM*. 3857–3866.
[77] Derong Xu, Ziheng Zhang, Zhenxi Lin, Xian Wu, Zhihong Zhu, Tong Xu, Xiangyu Zhao, Yefeng Zheng, and Enhong Chen. 2024. Multi-perspective Improvement of Knowledge Graph Completion with Large Language Models. In *LREC/COLING*.
[78] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. (2024).
[79] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 Technical Report. *arXiv e-prints* (2024), arXiv–2412.
[80] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*. 2369–2380.
[81] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *ICLR*.

[82] Chuanyue Yu, Kuo Zhao, Yuhan Li, Heng Chang, Mingjian Feng, Xiangzhe Jiang, Yufei Sun, Jia Li, Yuzhi Zhang, Jianxin Li, et al. 2025. GraphRAG-R1: Graph Retrieval-Augmented Generation with Process-Constrained Reinforcement Learning. *arXiv preprint arXiv:2507.23581* (2025).

[83] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. 2025. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476* (2025).

[84] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Rankrag: Unifying context ranking with retrieval-augmented generation in llms. *NeurIPS* 37 (2024), 121156–121184.

[85] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. 2023. Scaling Relationship on Learning Mathematical Reasoning with Large Language Models. *arXiv e-prints* (2023), arXiv–2308.

[86] Zhenrui Yue, Honglei Zhuang, Aijun Bai, Kai Hui, Rolf Jagerman, Hansi Zeng, Zhen Qin, Dong Wang, Xuanhui Wang, and Michael Bendersky. [n. d.]. Inference Scaling for Long-Context Retrieval Augmented Generation. In *ICLR*.

[87] Zhiyuan Zeng, Qinyuan Cheng, Zhangyue Yin, Bo Wang, Shimin Li, Yunhua Zhou, Qipeng Guo, Xuanjing Huang, and Xipeng Qiu. 2024. Scaling of search and learning: A roadmap to reproduce o1 from reinforcement learning perspective. *arXiv preprint arXiv:2412.14135* (2024).

[88] Wenlin Zhang, Xiangyang Li, Kuicai Dong, Yichao Wang, Pengyue Jia, Xiaopeng Li, Yingyi Zhang, Derong Xu, Zhaocheng Du, Huifeng Guo, et al. 2025. Process vs. Outcome Reward: Which is Better for Agentic RAG Reinforcement Learning. *arXiv preprint arXiv:2505.14069* (2025).

[89] Yingyi Zhang, Pengyue Jia, Derong Xu, Yi Wen, Xianneng Li, Yichao Wang, Wenlin Zhang, Xiaopeng Li, Weinan Gan, Huifeng Guo, et al. 2026. Personalize before retrieve: Llm-based personalized query expansion for user-centric retrieval. In *AAAI*, Vol. 40. 16406–16414.

[90] Yingyi Zhang, Junyi Li, Wenlin Zhang, Pengyue Jia, Xianneng Li, Yichao Wang, Derong Xu, Yi Wen, Huifeng Guo, Yong Liu, et al. [n. d.]. Evoking User Memory: Personalizing LLM via Recollection-Familiarity Adaptive Retrieval. In *The Fourteenth International Conference on Learning Representations*.

[91] Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. 2025. Knowledge Graph-Guided Retrieval Augmented Generation. In *NAACL*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 8912–8924.